

CRAGS

Technical Documentation

Compute Resource Allocation and Governance System

Architecture · Operations · API · User Workflows

Generated: 2026-06-06 18:19 UTC

CRAGS Technical Documentation

- [Repository Overview](#)
- [CRAGS](#)
 - [Features](#)
 - [Quick Start](#)
 - [Documentation](#)
 - [Repository Structure](#)
 - [Technology Stack](#)
 - [Backend Modules](#)
 - [Backend Module Documentation](#)
 - [Development Workflow](#)
 - [Common Commands](#)
 - [Default Credentials](#)
 - [Acknowledgements](#)
 - [License](#)
- [Documentation Index](#)
- [CRAGS Documentation](#)
 - [Start Here](#)
 - [Quick Links](#)
 - [Documentation Structure](#)
 - [Repository Layout](#)
 - [New Modules \(added in recent release\)](#)
- [System Architecture](#)
- [Architecture](#)
 - [Service Topology](#)
 - [Backend Layout](#)
 - [Frontend Layout](#)
 - [Data Model](#)
 - [Database Connection Pool](#)
 - [Caching](#)
 - [Auth and Roles](#)
 - [Policy Engine](#)
 - [Webhook Delivery](#)
- [Local Development](#)
- [Local Development](#)
 - [Prerequisites](#)
 - [One-Time Setup](#)
 - [Run The Full Stack](#)
 - [Login](#)
 - [Docker Compose Actions](#)
 - [Backend-Only Development](#)
 - [Frontend-Only Development](#)
 - [Migrations](#)
 - [Useful Checks](#)
- [Testing And Quality](#)
- [Testing And Quality](#)

- Backend Tests
 - Frontend Checks
 - Repo-Level Helper
 - What Is Not Covered
 - Suggested Developer Workflow
- Operations
- Operations
 - Runbook
 - Health Checks
 - Environment Variables
 - Redis (Rate Limiting and Cache)
 - Migrations
 - Super Admin Bootstrap
 - Troubleshooting
- API Reference
- API Reference
 - Authentication
 - System Endpoints
 - Booking Endpoints
 - IAM Endpoints
 - Audit Endpoints
 - Policy Endpoints
 - Maintenance Window Endpoints
 - Waitlist Endpoints
 - Analytics Endpoints
 - Webhook Endpoints
 - Billing Endpoints
 - Template Endpoints
 - System Health Endpoints
- Authentication Service
- Authentication Service
 - Table of Contents
 - Overview
 - Token Model
 - API Endpoints
 - RBAC Matrix
 - Login Rate Limiting
 - Token Blacklist
 - Password Reset Flow
 - Audit Trail
 - Frontend Integration
 - Configuration Reference
 - Database Schema
 - Operations
- Resources Service
- Resources Service
 - Table of Contents
 - Overview
 - Data Model
 - API Endpoints

- RBAC
 - SystemStatus Enforcement
 - Capacity Guard on Update
 - Audit Trail
 - Database Schema
- Notifications And Webhooks
- Notifications Service
 - Table of Contents
 - Overview
 - Architecture
 - Email Templates
 - Trigger Points
 - Password Reset Flow Change
 - Configuration Reference
 - Dev vs Production Mode
 - SMTP Compatibility
 - Adding a New Email
 - Module Layout
- Scheduling Engine
- Scheduler
 - Table of Contents
 - Overview
 - Jobs
 - Reconcile Algorithm
 - Concurrency and Locking
 - Lifecycle
 - Health Check
 - Configuration Reference
 - Scaling Roadmap
 - Future Algorithm Improvements
 - Operations
- User Manual
- CRAGS User Manual
 - Table of Contents
 - Getting Started
 - Logging In
 - Understanding the Dashboard
 - Viewing Compute Systems
 - Creating a Booking Request
 - Managing Your Bookings
 - Understanding Booking Status
 - Audit Trail
 - Team Management (Group Leads)
 - Resource Quotas and Limits
 - Troubleshooting
- Backend Guide
- CRAGS Backend
 - Compute Resource Allocation and Governance System (CRAGS)
- 1. System Overview
- 2. Technology Stack

- 3. Directory Structure
- 4. Core Architecture
- 5. Core Modules
 - 5.1 IAM (Identity and Access Management)
 - 5.2 Resources Module
 - 5.3 Scheduling Module
 - 5.4 Audit Module
- 6. Database Layer
- 7. Database Migrations
- 8. Running Database Migrations
- 9. Booking Conflict Prevention
- 10. Installation
 - Prerequisites
 - Setup Environment
- 11. Running the Backend
- 12. Docker Setup (Optional)
- 13. Development Workflow
- 14. Logging and Observability
- 15. Security Considerations
- 16. Future Extensions
- 17. Contribution Guidelines
- 18. License
- 19. Maintainers
- Frontend Guide
- CRAGS Frontend
 - Compute Resource Allocation and Governance System (CRAGS)
- 1. Technology Stack
- 2. Directory Structure
- 3. Application Shell and Routing
- 4. Data Flow
- 5. Installation
 - Prerequisites
 - Install Dependencies
- 6. Running, Building, and Linting
- 7. Design System
- 8. Contribution Guidelines
- 9. License

Repository Overview

CRAGS

Compute Resource Allocation and Governance System

CRAGS is a full-stack platform for managing, scheduling, and auditing institutional compute resources (GPUs, CPUs, and shared computational infrastructure). It enables research groups to reserve resources fairly while maintaining transparency and institutional oversight.

Features

- **Resource Scheduling:** Book GPU/CPU clusters with conflict prevention
- **Fair Allocation:** Enforce group quotas and usage limits
- **Prioritization:** Support foreground and background workload prioritization
- **Governance:** Comprehensive audit logging of all resource decisions
- **User Interface:** Modern web UI for system management and booking
- **Explainable Decisions:** Clear feedback on scheduling conflicts

Quick Start

Prerequisites

- Docker and Docker Compose (recommended)
- OR: Python 3.10+, Node.js 18+, PostgreSQL

Docker (Recommended)

```
# Clone and setup environment
git clone https://github.com/kushagrathisside/crags
cd crags
cp .env.example .env

# Edit .env: set JWT_SECRET_KEY and SUPERADMIN_PASSWORD

# Run the full stack
docker compose up --build
```

Access the application at: - **Frontend:** `http://localhost:5173` - **Backend API:** `http://localhost:8000` - **API Docs:** `http://localhost:8000/docs`

Local Development

For detailed local development setup without Docker, see [Local Development](#).

Documentation

Guide	Purpose
Local Development	Setup, prerequisites, environment configuration
Architecture	System design, backend modules, frontend structure, data model
API Reference	REST endpoints, authentication, request/response formats
Operations	Docker Compose runbook, health checks, troubleshooting
User Manual	End-user guide for using CRAGS

Repository Structure

```
crags/
├─ backend/           FastAPI service, SQLAlchemy ORM, Alembic migrations
├─ frontend/          React + TypeScript, Vite, UI components and hooks
├─ docs/              Project documentation
├─ docker-compose.yml  Local orchestration (PostgreSQL, backend, frontend)
├─ run.sh             Convenient Docker Compose wrapper
└─ README.md          This file
```

Technology Stack

Layer	Technology
Frontend	React, TypeScript, Vite, React Query, MUI
Backend	FastAPI, Python 3.10+, SQLAlchemy ORM
Database	PostgreSQL 15 with Alembic migrations
Deployment	Docker Compose with Nginx reverse proxy

Backend Modules

Module	Responsibility
IAM	Authentication, users, groups, roles, quotas
Resources	Compute system registration and inventory
Scheduling	Booking lifecycle, availability, conflict detection
Audit	Audit logging and usage tracking

Backend Module Documentation

Detailed backend architecture is available in [backend/README_backend.md](#).

Detailed frontend architecture is available in [frontend/README_frontend.md](#).

Development Workflow

```
# Terminal 1: Start the full stack
docker compose up --build
```

```
# The application automatically includes PostgreSQL, migrations, backend, and frontend
```

For backend-only or frontend-only development: - **Backend only:** See [Local Development - Backend](#) - **Frontend only:** See [Local Development - Frontend](#)

Common Commands

```
# Start full stack in foreground
./run.sh up

# Start detached
./run.sh upd

# View logs
./run.sh logs

# Stop services
./run.sh down

# Reset database
./run.sh reset
```

Or use Docker Compose directly:

```
docker compose up --build
docker compose down
docker compose ps
```

Default Credentials

The system creates a superadmin on startup when `SUPERADMIN_PASSWORD` is set in `.env`:

- **Username:** `superadmin`
- **Email:** `superadmin@crag.s.local`
- **Password:** Value from `SUPERADMIN_PASSWORD` in `.env` (default: `change-me`)

Acknowledgements

CRAGS was built with assistance from two AI systems, each playing a distinct role:

- **OpenAI Codex** — responsible for the initial architecture and the bulk of the implementation: backend modules, data models, scheduling logic, migrations, and the original frontend structure. The foundation of this project is Codex's work.
- **Anthropic Claude** — contributed refinements in a later session: the authentication layer, Redis caching, frontend redesign (design system, Gantt dashboard), bug fixes, and documentation cleanup.

Both tools were used under the direction of the project author, who designed the system, defined requirements, and made all architectural decisions.

License

See [LICENSE](#)

Documentation Index

CRAGS Documentation

CRAGS is the Compute Resource Allocation and Governance System. It provides a Docker-first full-stack application for registering compute systems, requesting resource bookings, enforcing group policy, and auditing scheduling decisions.

Start Here

- **New User?** → [User Manual](#): login, bookings, waitlist, analytics, and admin tasks.
- **Local Development**: run the stack, configure `.env`, and use backend/frontend dev commands.
- **Architecture**: service layout, backend modules, frontend structure, auth, data model, and scheduling.
- **API Reference**: all REST endpoints, request payloads, response shapes, and permission notes.
- **Auth Service**: token model, refresh flow, RBAC matrix, rate limiting, and password reset.
- **Resources Service**: compute system registry, CRUD, status enforcement, capacity guard, health monitoring, and maintenance windows.
- **Notifications Service**: SMTP email delivery, templates, trigger points, and dev mode.
- **Scheduler**: booking lifecycle, reconcile algorithm, approval workflow, waitlist promotion, SKIP LOCKED concurrency, and scaling notes.
- **Operations**: Docker Compose actions, health checks, migrations, environment variables, and troubleshooting.

Quick Links

Service	URL
Frontend	<code>http://localhost:5173</code>
Backend API	<code>http://localhost:8000</code>
OpenAPI docs	<code>http://localhost:8000/docs</code>
Health check	<code>http://localhost:8000/healthz</code>

Documentation Structure

Document	Audience	Purpose
UserManual.md	End users	Login, bookings, waitlist, analytics, admin queue
local-development.md	Developers	Dev environment, stack commands
architecture.md	Developers	System design, module map, data model
api-reference.md	Developers	Full REST endpoint reference
auth-service.md	Developers	Token lifecycle, RBAC, rate limiting
resources-service.md	Developers / DevOps	Compute system CRUD, health monitor, maintenance windows
notifications-service.md	Developers / DevOps	SMTP email, templates, dev mode
scheduler.md	Developers / DevOps	Booking engine, approval, waitlist, preemption
operations.md	DevOps / Admins	Docker Compose runbook, migrations, troubleshooting

Repository Layout

.	
└─ backend/	FastAPI service, SQLAlchemy models, Alembic migrations
└─ frontend/	React, TypeScript, Vite, MUI, React Query
└─ docs/	Project documentation
└─ docker-compose.yml	Local full-stack orchestration
└─ README.md	Project quick start

New Modules (added in recent release)

Module	Backend path	Frontend page
Policy Engine	<code>modules/policies/</code>	— (server-side enforcement)
Booking Approval	<code>modules/scheduling/</code> — approve/reject endpoints	Monitoring → Approval Queue
Booking Modifications	<code>modules/scheduling/</code> — extend/resize endpoints	Scheduler → Modify Booking
Maintenance Windows	<code>modules/maintenance/</code>	Systems → Maintenance Windows
Waitlist / Queue	<code>modules/waitlist/</code>	Scheduler → Waitlist

Module	Backend path	Frontend page
Analytics & Reporting	<code>modules/analytics/</code>	Analytics page
Webhooks	<code>modules/webhooks/</code>	— (admin REST only)
Audit Export	<code>modules/audit/</code> — CSV export	Monitoring
Cost / Billing	<code>modules/billing/</code>	— (REST only)
Booking Templates	<code>modules/templates/</code>	— (REST only)
System Health Monitor	<code>modules/health/</code>	— (auto-runs in reconciler)

System Architecture

Architecture

CRAGS is a full-stack scheduling and governance application built around a FastAPI API, PostgreSQL, Alembic migrations, and a React frontend.

Service Topology

`docker-compose.yml` defines four services:

Service	Purpose
<code>postgres</code>	PostgreSQL 15 database with a persistent Docker volume.
<code>migrate</code>	One-shot Alembic job that runs <code>alembic upgrade head</code> .
<code>backend</code>	FastAPI API process on port 8000 with APScheduler background worker.
<code>frontend</code>	Nginx serving the built React app and proxying <code>/api</code> to the backend.

Startup ordering is health-gated: PostgreSQL must be healthy before migrations, migrations must complete before the API starts.

Backend Layout

The backend source lives under `backend/src/crag`s.

Path	Responsibility
<code>main.py</code>	FastAPI app setup, CORS, <code>/healthz</code> , super-admin bootstrap, scheduler wiring.
<code>api/router.py</code>	Composes all module routers into one FastAPI router.
<code>core/config.py</code>	Environment-backed settings (pydantic-settings).
<code>core/security.py</code>	Password hashing and JWT token helpers.
<code>db/</code>	SQLAlchemy base, session factory, and connection pool.
<code>modules/iam/</code>	Users, groups, roles, login, quotas, RBAC dependencies.

Path	Responsibility
<code>modules/resources/</code>	Compute system registration and inventory.
<code>modules/scheduling/</code>	Booking lifecycle: create, cancel, approve, reject, extend, resize, preemption.
<code>modules/audit/</code>	Audit log model, listing, filtering, and CSV export.
<code>modules/notifications/</code>	SMTP email delivery and templates.
<code>modules/policies/</code>	Booking policy CRUD and enforcement (duration, concurrency, approval thresholds).
<code>modules/maintenance/</code>	Scheduled maintenance windows; auto-transitions systems to MAINTENANCE.
<code>modules/waitlist/</code>	Waitlist queue; promotes entries to bookings when capacity frees.
<code>modules/analytics/</code>	Usage breakdown per user/group/system; CSV export.
<code>modules/webhooks/</code>	HMAC-signed HTTP webhook delivery on booking events.
<code>modules/billing/</code>	Per-system cost rates; booking cost calculation and user summaries.
<code>modules/templates/</code>	Saved booking configuration templates per user.
<code>modules/health/</code>	HTTP health checks for compute systems; auto-MAINTENANCE on failure.

Reconciler Loop

`main.py` registers a single APScheduler interval job (`reconcile_bookings`) that runs every `RECONCILE_INTERVAL_MINUTES` (default 5) and calls, in order:

1. `scheduling.cron.reconcile_bookings` — expire/complete stale bookings, SKIP LOCKED.
2. `maintenance.service.apply_maintenance_transitions` — activate/deactivate windows.
3. `waitlist.service.promote_eligible` — convert WAITING entries to CONFIRMED bookings.
4. `health.service.run_health_checks` — ping `health_check_url` on each system; auto-transition status.

Frontend Layout

The frontend source lives under `frontend/src`.

Path	Responsibility
<code>App.tsx</code>	Authenticated router; role-gated routes via <code>RequireRole</code> .
<code>main.tsx</code>	React root with QueryClient, BrowserRouter, AppThemeProvider.
<code>api/client.ts</code>	Axios instance with 401-auto-refresh interceptor.
<code>api/cragsApi.ts</code>	Typed wrappers for all backend endpoints.
<code>hooks/</code>	React Query hooks for auth, systems, bookings, audit, availability, and mutations.
<code>context/ThemeContext.tsx</code>	Dark/light mode with localStorage persistence.
<code>theme/index.ts</code>	Centralised design system: <code>buildTheme(mode)</code> MUI factory plus exported <code>BRAND</code> palette, <code>STATUS_COLOR / statusTone()</code> , <code>CHART_COLORS</code> , and <code>FONT_SANS / FONT_MONO</code> (Roboto + Roboto Mono). Single source of truth for colour, typography, and shape.
<code>pages/</code>	One file per route: Dashboard, Scheduler, Systems, Monitoring, Analytics, Team, Login.
<code>components/layout/</code>	AppShell, Sidebar (collapsible), TopBar.
<code>components/forms/</code>	Booking request form, system registration form.

Path	Responsibility
<code>components/panels/</code>	Mission control, audit trail, booking lifecycle, system inventory, approval queue, waitlist, maintenance windows, booking actions.
<code>components/charts/</code>	Resource constraint chart, temporal Gantt.
<code>types/crags.ts</code>	Shared TypeScript types for all entities.
<code>utils/</code>	Local booking simulation helpers.

Frontend Routes

Route	Access	Page
<code>/</code>	Any authenticated	Dashboard (mission control)
<code>/scheduler</code>	Any authenticated	Booking request, Gantt, waitlist, modify bookings
<code>/systems</code>	RESOURCE_ADMIN, SUPER_ADMIN	System inventory, registration, maintenance windows
<code>/monitoring</code>	GROUP_LEAD+	Approval queue, audit trail, booking lifecycle
<code>/analytics</code>	GROUP_LEAD+	Usage charts, group table, CSV export
<code>/team</code>	RESOURCE_ADMIN, SUPER_ADMIN	Team management
<code>/login</code>	Public	Login form

Design System

All visual styling is centralised in `frontend/src/theme/index.ts` so a palette or typography change touches one file rather than every component:

Export	Purpose
<code>buildTheme(mode)</code>	MUI theme factory for <code>"light"</code> / <code>"dark"</code> ; flat bordered cards (elevation 0), 8px radius, no gradients or glow effects.
<code>BRAND</code>	Google Material palette — <code>blue #1A73E8</code> , <code>green #1E8E3E</code> , <code>amber #F9AB00</code> , <code>red #D93025</code> , <code>purple #9334E6</code> , <code>teal #12A4AF</code> .
<code>STATUS_COLOR</code> / <code>statusTone(status)</code>	Maps booking and system statuses (<code>ACTIVE</code> , <code>CONFIRMED</code> , <code>WAITING</code> , <code>PREEMPTED</code> , ...) to a <code>{ color, bg }</code> tone with a neutral fallback. Panels call <code>statusTone()</code> instead of hardcoding hex.
<code>CHART_COLORS</code>	Ordered series colours for the analytics Recharts views.
<code>FONT_SANS</code> / <code>FONT_MONO</code>	Roboto and Roboto Mono (Google's font stack), loaded via <code>index.html</code> .

Components consume semantic tokens (`primary.main`, `text.secondary`, `action.hover`, `divider`) and the exports above. Theme mode is persisted to `localStorage` by `context/ThemeContext.tsx` and defaults to light.

Data Model

Core tables (original)

Table	Purpose
<code>compute_systems</code>	Named compute systems with CPU, RAM, GPU, VRAM, type, status, and health fields.
<code>groups</code>	Research groups with concurrent and monthly resource quotas.
<code>users</code>	Local users with role, group membership, password hash, and activity state.
<code>bookings</code>	Resource requests over a PostgreSQL <code>TSRANGE</code> booking period; includes approval fields.
<code>audit_logs</code>	All system events with action, record_id, user_id, and timestamp.

New tables (added in recent release)

Table	Purpose
<code>booking_policies</code>	Named policy sets: duration limits, concurrency caps, approval thresholds.
<code>maintenance_windows</code>	Scheduled maintenance windows per system (start_time, end_time, reason).
<code>waitlist_entries</code>	Queued capacity requests; promoted to bookings by reconciler.
<code>webhooks</code>	Registered webhook endpoints with event subscriptions and HMAC secret.
<code>system_costs</code>	Per-system resource hour rates (CPU, GPU, RAM, VRAM).
<code>booking_templates</code>	Saved booking configurations per user.

Booking approval fields (added to `bookings`)

Column	Type	Purpose
<code>approved_by</code>	<code>integer FK → users.id</code>	Admin who approved/rejected.
<code>approved_at</code>	<code>datetime</code>	When the approval decision was made.
<code>rejection_reason</code>	<code>text</code>	Reason text for rejected bookings.

System health fields (added to `compute_systems`)

Column	Type	Purpose
<code>health_check_url</code>	<code>text</code>	HTTP endpoint to ping.
<code>last_health_check_at</code>	<code>datetime</code>	Timestamp of most recent check.
<code>last_health_status</code>	<code>varchar(20)</code>	<code>UP</code> , <code>DOWN</code> , or <code>SKIP</code> .

Bookings use a GiST index on `booking_period` for efficient overlap queries.

Database Connection Pool

Configured in `db/session.py` via SQLAlchemy's `create_engine`:

Setting	Value	Effect
<code>pool_size</code>	10	Minimum persistent connections kept open.
<code>max_overflow</code>	20	Extra connections allowed during traffic spikes (30 total max).
<code>pool_pre_ping</code>	<code>True</code>	Validates each connection with <code>SELECT 1</code> before use; silently replaces stale ones.
<code>pool_recycle</code>	3600 s	Forces connection replacement every hour, preventing server-side timeouts from PgBouncer, RDS idle-client timeouts, or NAT gateway resets.
<code>autocommit</code>	<code>False</code>	All writes require an explicit <code>db.commit()</code> .
<code>autoflush</code>	<code>False</code>	ORM changes are not flushed to the DB until commit or an explicit flush.

Sessions are **request-scoped**: `get_db()` opens one session per HTTP request and closes it in the `finally` block regardless of success or failure. The reconciler sub-tasks each use their own independently managed sessions to prevent state leakage between tasks.

Caching

CRAGS uses a lightweight two-backend cache (`core/cache.py`) that mirrors the rate-limiter pattern:

Backend	When active	Storage
<code>_RedisCache</code>	<code>REDIS_URL</code> is set and reachable at startup	JSON in Redis keys with TTL via <code>SETEX</code>
<code>_InMemoryCache</code>	Redis not configured or unreachable	Python dict with monotonic-clock expiry, thread-safe via <code>threading.Lock</code>

The instance is resolved once (double-checked locking) and reused for the lifetime of the process.

What is cached

Cache key	TTL	Endpoint	Invalidated by
<code>craggs:cache:systems</code>	60 s	<code>GET /api/v1/systems/</code> (unfiltered)	<code>POST/PATCH/DELETE /systems/</code> , health-check reconciler
<code>craggs:cache:policies</code>	300 s	<code>GET /api/v1/policies</code>	<code>POST/PATCH/DELETE /policies/{id}</code>
<code>craggs:cache:billing:costs</code>	300 s	<code>GET /api/v1/billing/costs</code>	<code>PUT /billing/costs/{system_id}</code>

Only the **unfiltered** systems list is cached; queries with `?status=` hit the database directly. Filtered queries are infrequent admin operations where freshness matters.

Invalidation strategy

Writes call `get_cache().delete(key)` immediately after committing the database transaction. This is **cache-aside with write-invalidation** (not write-through): the next read repopulates the cache from the database. The health-check

reconciler automatically busts `craggs:cache:systems` whenever it transitions any system's status.

What is not cached

Booking availability, booking lists, audit logs, and all user-scoped queries are **never cached** — stale reads on booking data would cause incorrect capacity calculations and double-bookings.

The token blacklist lookup (one DB read per authenticated request) is also not cached here. At high request volume, that lookup is the next candidate to move to Redis with a 15-minute TTL matching the access token lifetime.

Auth and Roles

Authentication is local username/email + password. Successful login sets an HTTP-only cookie. Bearer token is also accepted for non-browser clients.

Role	Access
MEMBER	Own bookings, scheduler, dashboard.
GROUP_LEAD	Own bookings + group member bookings, monitoring, analytics.
RESOURCE_ADMIN	Full booking scope, systems, team management, approval queue.
SUPER_ADMIN	All of the above plus user/group creation.

Policy Engine

When `create_booking` is called, the service resolves the applicable `BookingPolicy` for the user's group (group-scoped first, then global default). The policy can:

- **Block** bookings that exceed hard limits (`max_duration_hours`, `max_advance_days`, `max_concurrent_bookings`).
- **Route** bookings to `REQUESTED` (pending admin approval) when resource thresholds are exceeded or `always_require_approval` is set.

Admins approve or reject `REQUESTED` bookings via `PATCH /bookings/{id}/approve` and `PATCH /bookings/{id}/reject`.

Webhook Delivery

When booking state changes occur, `webhooks.service.emit_event` is called. It queries all active webhooks subscribed to the event (or `"*"`), then fires a HMAC-SHA256-signed HTTP POST in a daemon thread per webhook. The `secret` field is used to compute `X-CRAGS-Signature: sha256=<hex>` so consumers can verify authenticity. Delivery is best-effort; failures are logged and `last_status_code` is recorded.

Local Development

Local Development

The project is Docker-first. Use Docker Compose for the normal full-stack local environment, because that path includes PostgreSQL, migrations, backend startup, frontend serving, proxying, and health checks.

Prerequisites

- Docker Desktop or Docker Engine
- Docker Compose v2 via `docker compose`, or the legacy `docker-compose` binary
- At least 4 GB of memory available to Docker

- Optional for backend-only work: `uv`
- Optional for frontend-only work: Node.js and npm

On WSL, enable Docker Desktop integration for the distro that contains this repository.

One-Time Setup

From the repository root:

```
cp .env.example .env
```

Edit `.env` before regular use:

- Set `JWT_SECRET_KEY` to a unique local value.
- Set `SUPERADMIN_PASSWORD` to a secure value.
- Keep `DATABASE_URL` pointed at the Compose service host `postgres` when using Docker Compose.

The default host ports are:

Variable	Default	Service
<code>POSTGRES_PORT</code>	<code>5432</code>	PostgreSQL
<code>BACKEND_PORT</code>	<code>8000</code>	FastAPI
<code>FRONTEND_PORT</code>	<code>5173</code>	React app served by Nginx

Run The Full Stack

From the repository root:

```
docker-compose up --build
```

Detached mode:

```
docker-compose up --build -d
```

Access the app at `http://localhost:5173`. The backend OpenAPI UI is available at `http://localhost:8000/docs`.

Login

On backend startup, CRAGS bootstraps a super admin user if `SUPERADMIN_PASSWORD` is non-empty.

Use the values from `.env`:

- Username: `SUPERADMIN_USERNAME`
- Password: `SUPERADMIN_PASSWORD`

The bootstrap also creates or reuses `SUPERADMIN_GROUP_NAME`.

Docker Compose Actions

```
docker-compose up --build           # Build and start in foreground
docker-compose up --build -d        # Build and start detached
docker-compose logs -f              # Follow service logs
docker-compose ps                   # Show service status
docker-compose down --remove-orphans # Stop and remove containers
docker-compose down --remove-orphans --volumes # Stop stack and remove database volume
```


`reset` deletes the PostgreSQL Docker volume for this project.

Backend-Only Development

Use this path when you already have PostgreSQL available locally and want a reloading FastAPI process outside Docker:

```
cd backend
uv sync
export DATABASE_URL=postgresql+psycopg://crag:crag@localhost:5432/crag
export PYTHONPATH=src
uv run alembic upgrade head
uv run uvicorn crags.main:app --reload --app-dir src --host 0.0.0.0 --port 8000
```

The backend reads settings from environment variables and from `.env` according to `pydantic-settings`.

Frontend-Only Development

Start the backend first, then run Vite:

```
cd frontend
npm install
npm run dev
```

The Vite dev server listens on `0.0.0.0:5173` and proxies `/api` requests to `http://localhost:8000`.

The frontend API client defaults to `/api/v1`. Override with `VITE_API_BASE_URL` when needed.

Migrations

Docker Compose runs migrations automatically through the `migrate` service.

For local backend-only work:

```
cd backend
uv run alembic upgrade head
```

Create a new migration after model changes:

```
cd backend
uv run alembic revision --autogenerate -m "describe change"
```

Review generated migrations before applying them, especially PostgreSQL enum, range, and index changes.

Useful Checks

Backend:

```
cd backend
uv run pytest
uv run ruff check .
uv run black --check .
```

Frontend:

```
cd frontend
npm run lint
npm run build
```

There is no single repo-level test command at the moment.

Testing And Quality

Testing And Quality

This guide documents the runnable checks in the repository and the environment they expect.

Backend Tests

The backend suite lives under `backend/tests/` and currently covers API, IAM, scheduling, resources, and integration flows.

Run from `backend/`:

```
PYTHONPATH=src ./venv/bin/python -m pytest tests -q
```

Requirements

- PostgreSQL must be reachable.
- If `CRAGS_TEST_DATABASE_URL` is unset, the test fixtures default to:

```
postgresql+psycopg://crag:crag@127.0.0.1:5433/crag
```

Practical Notes

- Running tests from the repo root without `PYTHONPATH=src` will fail module imports.
- In this environment, local PostgreSQL access from inside the sandbox was blocked, but the test suite passed outside the sandbox.

Frontend Checks

From `frontend/`:

```
npm run build
npm run lint
```

Current observed state:

- `npm run build` succeeds.
- `npm run lint` currently reports real issues, mainly React purity rules, fast-refresh export constraints, and one explicit `any`.

Repo-Level Helper

The repository includes `test_all.sh`, which is intended as a convenience wrapper for backend checks and can be extended further if stricter repo-level automation is added later.

What Is Not Covered

- There is no frontend test suite configured in `package.json`.
- There is no CI configuration in this repository snapshot.
- Compose startup validation is operational rather than unit-tested.

Suggested Developer Workflow

1. Run `docker compose up -d postgres` or the full stack.
2. Run backend tests.
3. Run frontend build.
4. Run frontend lint and address remaining issues before release work.

Operations

Operations

This guide covers the project-standard local operations flow. For most day to day work, use Docker Compose directly from the repository root.

Runbook

Start in foreground:

```
docker-compose up --build
```

Start detached:

```
docker-compose up --build -d
```

Show service status:

```
docker-compose ps
```

Tail logs:

```
docker-compose logs -f
```

Stop services:

```
docker-compose down --remove-orphans
```

Reset containers and database volume:

```
docker-compose down --remove-orphans --volumes
```

After the volume reset, run `docker-compose up --build` to recreate the database, rerun migrations, and seed the super admin.

Health Checks

Backend health:

```
curl http://localhost:8000/healthz
```

Expected response:

```
{
  "status": "ok"
}
```

The Compose backend health check also calls this endpoint before dependent services continue.

Environment Variables

Local defaults are defined in `.env.example`.

Variable	Purpose	Default
<code>POSTGRES_PORT</code>	Host port mapped to PostgreSQL.	<code>5432</code>
<code>BACKEND_PORT</code>	Host port mapped to FastAPI.	<code>8000</code>
<code>FRONTEND_PORT</code>	Host port mapped to Nginx frontend.	<code>5173</code>
<code>POSTGRES_DB</code>	Database name.	<code>crag</code>
<code>POSTGRES_USER</code>	Database user.	<code>crag</code>
<code>POSTGRES_PASSWORD</code>	Database password.	<code>crag</code>
<code>DATABASE_URL</code>	SQLAlchemy database URL.	Compose URL using host <code>postgres</code>
<code>JWT_SECRET_KEY</code>	Secret used to sign JWTs.	<code>change-me-local-dev-secret</code>
<code>JWT_ALGORITHM</code>	JWT signing algorithm.	<code>HS256</code>
<code>ACCESS_TOKEN_EXPIRE_MINUTES</code>	Auth session lifetime.	<code>60</code>
<code>FRONTEND_ORIGINS</code>	Comma-separated CORS origins.	<code>http://localhost:5173</code>
<code>AUTH_COOKIE_NAME</code>	Session cookie name.	<code>crag_session</code>
<code>AUTH_COOKIE_SECURE</code>	Restrict cookie to HTTPS.	<code>false</code>
<code>AUTH_COOKIE_SAMESITE</code>	Cookie SameSite policy.	<code>lax</code>
<code>AUTH_COOKIE_DOMAIN</code>	Optional cookie domain.	<code>unset</code>
<code>SUPERADMIN_USERNAME</code>	Bootstrapped admin username.	<code>superadmin</code>
<code>SUPERADMIN_EMAIL</code>	Bootstrapped admin email.	<code>superadmin@crag.local</code>
<code>SUPERADMIN_PASSWORD</code>	Bootstrapped admin password.	<code>change-me</code>
<code>SUPERADMIN_GROUP_NAME</code>	Bootstrapped admin group.	<code>platform-admins</code>
<code>REDIS_URL</code>	Optional Redis connection string. Enables distributed login rate limiting and the response cache. When unset, both fall back to an in-process implementation (single-replica only). Example: <code>redis://redis:6379/0</code>	<code>unset</code>

Variable	Purpose	Default
<code>RECONCILE_INTERVAL_MINUTES</code>	How often the background reconciler runs (booking expiry, waitlist promotion, health checks).	5
<code>AUDIT_RETAIN_DAYS</code>	Audit log rows older than this are pruned by the reconciler.	90

For non-local deployments, use a strong `JWT_SECRET_KEY`, set `AUTH_COOKIE_SECURE=true` behind HTTPS, and configure `FRONTEND_ORIGINS` to the actual frontend origin.

Redis (Rate Limiting and Cache)

Redis is **optional**. Without it the application runs correctly using in-process fallbacks, but those fallbacks are single-replica only and reset on restart.

Add Redis to `docker-compose.yml`:

```
services:
  redis:
    image: redis:7-alpine
    restart: unless-stopped
    ports:
      - "6379:6379"
```

Then set in `.env`:

```
REDIS_URL=redis://redis:6379/0
```

With Redis configured:

Feature	Behaviour
Login rate limiter	Counters survive restarts; works across multiple API replicas.
Response cache	<code>GET /systems/</code> , <code>GET /policies</code> , <code>GET /billing/costs</code> served from Redis for their TTL (60 s / 300 s / 300 s). Invalidated automatically on writes.

Without Redis, both features fall back to in-memory and log a notice at startup:

```
Cache: in-memory (set REDIS_URL for distributed mode)
Rate limiter: in-memory (set REDIS_URL for distributed mode)
```

Migrations

Migrations are stored in `backend/migrations/versions`.

The Compose workflow runs:

```
alembic upgrade head
```

inside the `migrate` service before the API starts.

For manual migration work:

```
cd backend
uv run alembic upgrade head
uv run alembic current
uv run alembic history
```

When adding model changes, generate a migration, inspect it, then apply it:

```
cd backend
uv run alembic revision --autogenerate -m "describe change"
uv run alembic upgrade head
```

Super Admin Bootstrap

On backend startup, `ensure_super_admin` runs once through the FastAPI startup hook.

If `SUPERADMIN_PASSWORD` is blank, no bootstrap user is created. If it is set, the backend creates or updates the configured super admin and ensures that user belongs to `SUPERADMIN_GROUP_NAME`.

Changing the password for an existing bootstrapped user is not automatic unless the user has no stored password hash. Use the IAM APIs or admin UI to rotate an existing user's password.

Troubleshooting

Docker command not found

Install Docker Desktop or Docker Engine and ensure `docker` is on your shell `PATH`.

Compose plugin not available

Install Docker Compose v2, or install the legacy `docker-compose` binary.

Port already in use

Change the relevant host port in `.env`:

- `POSTGRES_PORT`
- `BACKEND_PORT`
- `FRONTEND_PORT`

Then restart:

```
docker-compose down --remove-orphans
docker-compose up --build
```

Login fails for the super admin

Confirm that `.env` contains a non-empty `SUPERADMIN_PASSWORD`. Then restart the backend:

```
docker-compose down --remove-orphans
docker-compose up --build
```

If the database already contains the user and you need a clean bootstrap, use:

```
docker-compose down --remove-orphans --volumes
docker-compose up --build
```

Migrations fail at startup

Inspect migration logs:

```
docker-compose logs -f
```

Common causes are an invalid `DATABASE_URL` , stale local volumes, or a migration that needs manual review before it can run against existing data.

Frontend cannot reach API

In Docker, Nginx proxies `/api/` to the `backend` Compose service. In Vite development, `frontend/vite.config.ts` proxies `/api` to `http://localhost:8000` .

Check that the backend is healthy and that the frontend is using the expected API base URL.

API Reference

API Reference

The backend serves REST endpoints from the FastAPI app. With default local ports, the base URL is `http://localhost:8000` .

Interactive OpenAPI docs are available at `http://localhost:8000/docs` when the backend is running. All application endpoints are under `/api/v1` .

Authentication

- POST `/api/v1/auth/login`
- POST `/api/v1/auth/logout`
- POST `/api/v1/auth/refresh`
- POST `/api/v1/auth/forgot-password`
- POST `/api/v1/auth/reset-password`
- GET `/api/v1/users/me`

Login sets an HTTP-only session cookie. Non-browser clients may also use `Authorization: Bearer <token>` . See [Auth Service](#) for full details.

System Endpoints

Method	Path	Auth	Description
GET	<code>/healthz</code>	None	Backend + scheduler health.
GET	<code>/api/v1/systems/</code>	Any	List all compute systems.
POST	<code>/api/v1/systems/</code>	Admin	Register a compute system.
PATCH	<code>/api/v1/systems/{id}</code>	Admin	Update system fields or status.
DELETE	<code>/api/v1/systems/{id}</code>	Admin	Set system OFFLINE (soft delete).

Create / update payload fields:

Field	Type	Notes
<code>name</code>	string	Unique system name.

Field	Type	Notes
<code>system_type</code>	<code>CPU</code> <code>GPU</code> <code>HYBRID</code>	
<code>cpu_cores</code>	integer	
<code>ram_gb</code>	integer	
<code>gpu_units</code>	integer	
<code>vram_gb</code>	integer	
<code>status</code>	<code>ACTIVE</code> <code>MAINTENANCE</code> <code>OFFLINE</code>	
<code>health_check_url</code>	string (optional)	HTTP endpoint for automatic health pings.

Booking Endpoints

Method	Path	Auth	Description
<code>GET</code>	<code>/api/v1/bookings/systems/{id}/availability</code>	Any	Available resources in a time window.
<code>GET</code>	<code>/api/v1/bookings/</code>	Scoped	List bookings (paginated).
<code>GET</code>	<code>/api/v1/bookings/{id}</code>	Scoped	Fetch one booking.
<code>POST</code>	<code>/api/v1/bookings/</code>	Any	Create a booking. May return <code>REQUESTED</code> if policy requires approval.
<code>PATCH</code>	<code>/api/v1/bookings/{id}/cancel</code>	Owner / Admin	Cancel a <code>REQUESTED</code> or <code>CONFIRMED</code> booking.
<code>PATCH</code>	<code>/api/v1/bookings/{id}/approve</code>	Admin	Approve a <code>REQUESTED</code> booking → <code>CONFIRMED</code> .
<code>PATCH</code>	<code>/api/v1/bookings/{id}/reject</code>	Admin	Reject a <code>REQUESTED</code> booking → <code>CANCELLED</code> .
<code>PATCH</code>	<code>/api/v1/bookings/{id}/extend</code>	Owner / Admin	Push <code>end_time</code> forward (capacity-checked).
<code>PATCH</code>	<code>/api/v1/bookings/{id}/resize</code>	Owner / Admin	Change <code>req_cpu/gpu/ram/vram</code> (capacity-checked).

Booking list query parameters

Parameter	Type	Notes
<code>system_id</code>	integer	
<code>user_id</code>	integer	Admins only for other users.
<code>status</code>	string	e.g. <code>REQUESTED</code> , <code>CONFIRMED</code> .
<code>academic_category</code>	string	
<code>start_time</code>	ISO datetime	

Parameter	Type	Notes
end_time	ISO datetime	
limit	integer (1–1000)	Default 200.
offset	integer	Default 0.

Create booking payload

```
{
  "system_id": 1,
  "start_time": "2026-06-01T09:00:00Z",
  "end_time": "2026-06-01T17:00:00Z",
  "req_cpu": 8,
  "req_gpu": 2,
  "req_ram": 64,
  "req_vram": 32,
  "access_type": "BACKGROUND",
  "academic_category": "Research",
  "project_title": "LLM fine-tune",
  "expected_deliverable": "Checkpoint + eval report",
  "objective": "Benchmark new LoRA adapter on vision tasks."
}
```

access_type: FOREGROUND (preempts background jobs) or BACKGROUND .

If a booking policy routes the request to approval, the response status field is REQUESTED instead of CONFIRMED . Admins then call /approve or /reject .

Reject payload

```
{ "reason": "Resource request exceeds project allocation." }
```

Extend payload

```
{ "new_end_time": "2026-06-01T21:00:00Z" }
```

Resize payload (all fields optional)

```
{ "req_cpu": 4, "req_gpu": 1, "req_ram": 32, "req_vram": 16 }
```

Conflict response (HTTP 409)

```
{
  "detail": "GPU capacity exceeded",
  "reason": "CAPACITY_EXCEEDED",
  "resource": "GPU",
  "shortage": 1,
  "overlap_window": { "start_time": "...", "end_time": "..." },
  "conflicting_booking_ids": [12, 13],
  "recommended_fixes": ["Reduce GPU request", "Choose another window"]
}
```

Known `reason` values: `CAPACITY_EXCEEDED`, `PREEMPTION_INSUFFICIENT`, `GROUP_CONCURRENT_QUOTA_EXCEEDED`, `GROUP_MONTHLY_QUOTA_EXCEEDED`, `INVALID_TRANSITION`.

IAM Endpoints

Method	Path	Auth	Description
GET	<code>/api/v1/iam/groups</code>	Admin	List all groups.
POST	<code>/api/v1/iam/groups</code>	Admin	Create a group.
PATCH	<code>/api/v1/iam/groups/{id}</code>	Admin	Update group name or quotas.
GET	<code>/api/v1/iam/users</code>	Admin	List users.
POST	<code>/api/v1/iam/users</code>	Admin	Create a user.
PATCH	<code>/api/v1/iam/users/{id}</code>	Admin	Update user settings.
GET	<code>/api/v1/groups/{id}/members</code>	Group lead / Admin	List group members.
GET	<code>/api/v1/groups/{id}/usage</code>	Group lead / Admin	Monthly usage summary (<code>? month=YYYY-MM</code>).

Group quota fields: `concurrent_cpu_quota`, `concurrent_gpu_quota`, `concurrent_ram_quota`, `concurrent_vram_quota`, `monthly_cpu_hours_quota`, `monthly_gpu_hours_quota`, `monthly_ram_gb_hours_quota`, `monthly_vram_gb_hours_quota`. Use `null` for unlimited.

Audit Endpoints

Method	Path	Auth	Description
GET	<code>/api/v1/audit/</code>	Admin	List audit events (paginated, filtered).
GET	<code>/api/v1/audit/actions</code>	Admin	List all valid audit action enum values.
GET	<code>/api/v1/audit/export.csv</code>	Admin	Download filtered audit log as CSV.

Audit query parameters

Parameter	Type	Notes
<code>limit</code>	integer (1–1000)	Default 200.
<code>offset</code>	integer	Default 0.
<code>table_name</code>	string	Filter by table.
<code>action</code>	string	e.g. <code>BOOKING_APPROVED</code> .
<code>user_id</code>	integer	
<code>from_time</code>	ISO datetime	Inclusive lower bound on timestamp.
<code>to_time</code>	ISO datetime	Inclusive upper bound on timestamp.

The CSV export accepts the same filters and returns a `text/csv` response.

Policy Endpoints

Method	Path	Auth	Description
GET	/api/v1/policies	Any	List all policies.
POST	/api/v1/policies	Admin	Create a policy.
GET	/api/v1/policies/{id}	Any	Fetch one policy.
PATCH	/api/v1/policies/{id}	Admin	Update a policy.
DELETE	/api/v1/policies/{id}	Admin	Delete a policy.

Policy payload fields

Field	Type	Effect
name	string	Unique policy name.
max_duration_hours	integer null	Hard limit on booking length. Exceeded → 422.
max_advance_days	integer null	Hard limit on how far ahead to book.
max_concurrent_bookings	integer null	Hard limit on active bookings per user.
approval_required_above_gpu	integer null	Route to REQUESTED if req_gpu exceeds this.
approval_required_above_cpu	integer null	Route to REQUESTED if req_cpu exceeds this.
approval_required_above_ram_gb	integer null	Route to REQUESTED if req_ram exceeds this.
approval_required_above_hours	integer null	Route to REQUESTED if duration exceeds this.
always_require_approval	boolean	Always route to REQUESTED regardless of resources.
group_id	integer null	Scope policy to a specific group (null = global).
is_default	boolean	Use as fallback when no group-scoped policy matches.

Maintenance Window Endpoints

Method	Path	Auth	Description
GET	/api/v1/maintenance	Any	List maintenance windows. Optional ? system_id=.
POST	/api/v1/maintenance	Admin	Schedule a maintenance window.
DELETE	/api/v1/maintenance/{id}	Admin	Delete a maintenance window.

Create maintenance window payload

```

{
  "system_id": 2,
  "start_time": "2026-06-10T02:00:00Z",
  "end_time": "2026-06-10T06:00:00Z",
  "reason": "Kernel upgrade and GPU driver update"
}

```

When a window becomes active, the reconciler sets the system to `MAINTENANCE` and cancels any overlapping `CONFIRMED` or `REQUESTED` bookings. When it ends, the system returns to `ACTIVE` (unless the health monitor reports it as `DOWN`).

Waitlist Endpoints

Method	Path	Auth	Description
<code>GET</code>	<code>/api/v1/waitlist</code>	Scoped	List waitlist entries (admins see all; others see own).
<code>POST</code>	<code>/api/v1/waitlist</code>	Any	Join the waitlist for a system.
<code>DELETE</code>	<code>/api/v1/waitlist/{id}</code>	Owner / Admin	Cancel a waitlist entry.

Join payload

```

{
  "system_id": 1,
  "req_cpu": 8,
  "req_gpu": 2,
  "req_ram": 64,
  "req_vram": 32,
  "duration_hours": 6,
  "access_type": "FOREGROUND",
  "academic_category": "Research",
  "project_title": "Fine-tune run"
}

```

The reconciler calls `promote_eligible()` on every tick. When a `WAITING` entry fits current capacity, it creates a `CONFIRMED` booking and sets the entry status to `PROMOTED`.

Analytics Endpoints

Method	Path	Auth	Description
<code>GET</code>	<code>/api/v1/analytics</code>	GROUP_LEAD+	Usage breakdown for a time window.
<code>GET</code>	<code>/api/v1/analytics/export.csv</code>	GROUP_LEAD+	Download usage breakdown as CSV.

Query parameters

Parameter	Type	Default
<code>from_time</code>	ISO datetime	30 days ago

Parameter	Type	Default
to_time	ISO datetime	now

Response structure

```
{
  "from_time": "...",
  "to_time": "...",
  "total_bookings": 142,
  "total_cpu_hours": 1280.5,
  "total_gpu_hours": 348.0,
  "total_ram_gb_hours": 81920.0,
  "total_vram_gb_hours": 11136.0,
  "per_user": [
    { "user_id": 3, "username": "alice", "cpu_hours": 128, "gpu_hours": 48,
      "ram_gb_hours": 8192, "vram_gb_hours": 1536, "booking_count": 5 }
  ],
  "per_group": [ ... ],
  "per_system": [
    { "system_id": 1, "system_name": "gpu-cluster-a", "cpu_utilization_pct": 62.5,
      "gpu_utilization_pct": 87.0, "ram_utilization_pct": 40.0, "vram_utilization_pct": 55.0,
      "booking_count": 22, "active_hours": 156.0 }
  ]
}
```

Webhook Endpoints

Method	Path	Auth	Description
GET	/api/v1/webhooks	Admin	List webhooks.
POST	/api/v1/webhooks	Admin	Create a webhook.
GET	/api/v1/webhooks/{id}	Admin	Fetch one webhook.
PATCH	/api/v1/webhooks/{id}	Admin	Update webhook fields.
DELETE	/api/v1/webhooks/{id}	Admin	Delete a webhook.

Create payload

```
{
  "name": "CI notification",
  "url": "https://hooks.example.com/crags",
  "events": ["BOOKING_APPROVED", "BOOKING_REJECTED", "BOOKING_PREEMPTED"],
  "secret": "my-hmac-secret",
  "active": true
}
```

Use "events": ["*"] to subscribe to all events. The backend signs each request with X-CRAGS-Signature: sha256=<hmac-hex> using the configured secret.

Billing Endpoints

Method	Path	Auth	Description
GET	/api/v1/billing/costs	Any	List per-system cost rates.
PUT	/api/v1/billing/costs/{system_id}	Admin	Upsert cost rates for a system.
GET	/api/v1/billing/bookings/{id}/cost	Any	Calculate cost for one booking.
GET	/api/v1/billing/summary	Admin	Per-user cost totals (optional <code>from_time</code> / <code>to_time</code>).

Cost rate payload

```
{
  "cpu_core_hour_rate": 0.02,
  "gpu_hour_rate": 0.50,
  "ram_gb_hour_rate": 0.005,
  "vram_gb_hour_rate": 0.01,
  "currency": "USD"
}
```

Template Endpoints

Method	Path	Auth	Description
GET	/api/v1/templates	Scoped	List templates (admins see all; others see own).
POST	/api/v1/templates	Any	Save a booking template.
GET	/api/v1/templates/{id}	Owner / Admin	Fetch one template.
PATCH	/api/v1/templates/{id}	Owner / Admin	Update a template.
DELETE	/api/v1/templates/{id}	Owner / Admin	Delete a template.

Template fields mirror `BookingCreate` but with `duration_hours` (integer) instead of `start_time` / `end_time`, plus an optional `name` (max 100 chars).

System Health Endpoints

Method	Path	Auth	Description
GET	/api/v1/health	Any	List health status for all systems (or <code>?system_id=</code>).
POST	/api/v1/health/run-checks	Any	Trigger health checks immediately.

The reconciler runs health checks automatically on every tick. Results are also stored in `last_health_check_at` and `last_health_status` on each compute system.

Authentication Service

Authentication Service

This document covers the full authentication and authorization layer added to CRAGS: token lifecycle, refresh flow, token revocation, login rate limiting, password reset, RBAC enforcement, and the frontend integration.

Table of Contents

- 1. Overview
- 2. Token Model
- 3. API Endpoints
- 4. RBAC Matrix
- 5. Login Rate Limiting
- 6. Token Blacklist
- 7. Password Reset Flow
- 8. Audit Trail
- 9. Frontend Integration
- 10. Configuration Reference
- 11. Database Schema
- 12. Operations

Overview

Authentication is local username/email + password. A successful login issues two HTTP-only cookies:

Cookie	TTL	Purpose
crag_session	15 minutes	Access token — sent with every API request
crag_refresh	7 days	Refresh token — sent only to POST /api/v1/auth/refresh

The short access token TTL limits the blast radius of a stolen cookie. The refresh token lets the frontend obtain a new access token transparently, so users never see a login prompt during a normal session.

Both tokens are JWTs signed with JWT_SECRET_KEY using HS256. Every token carries a jti (JWT ID, a UUID) used for revocation.

Token Model

Access token payload

```
{
  "sub": "1",
  "role": "MEMBER",
  "group_id": 3,
  "type": "access",
  "jti": "d4e5f6a7-...",
  "iat": 1748217600,
  "exp": 1748218500
}
```

Refresh token payload

```
{
  "sub": "1",
  "type": "refresh",
  "jti": "a1b2c3d4-...",
  "iat": 1748217600,
  "exp": 1748822400
}
```

Validation order in `get_current_user`

1. Resolve token from cookie (`crag_session`) or `Authorization: Bearer <token>` header.
2. Decode and verify signature + expiry.
3. Assert `type == "access"` — refresh tokens are rejected.
4. Assert `jti` is present.
5. Check `jti` is not in `token_blacklist`.
6. Load the `User` row; assert `is_active == True`.

API Endpoints

All auth endpoints are under `/api/v1/auth/`. No authentication is required to call them except where noted.

POST `/api/v1/auth/login`

Authenticate with username/email + password. Issues both cookies.

Request

```
{
  "identifier": "superadmin",
  "password": "change-me"
}
```

`identifier` may be replaced by `username` or `email`.

Response `200`

```
{
  "token_type": "cookie",
  "access_token_expires_at": "2026-05-26T12:15:00Z",
  "user": {
    "id": 1,
    "username": "superadmin",
    "email": "superadmin@crag.local",
    "role": "SUPER_ADMIN",
    "group_id": 1,
    "group_name": "platform-admins",
    "is_active": true,
    "auth_provider": "local",
    "created_at": "2026-05-01T00:00:00Z",
    "last_login": "2026-05-26T12:00:00Z"
  }
}
```

Error responses

Status	Condition
400	No identifier provided
401	Wrong credentials
429	Rate limit exceeded (5 failures in 10 minutes)

POST /api/v1/auth/refresh

Exchange the refresh cookie for a new access + refresh token pair. The old refresh token JTI is blacklisted (token rotation).

The refresh cookie is scoped to this path (`path=/api/v1/auth/refresh`), so browsers only send it to this endpoint.

Response `200` — same shape as login.

Error responses

Status	Condition
401	No refresh cookie, invalid/expired token, or token blacklisted

POST /api/v1/auth/logout

Revoke both cookies. Reads token JTIs from cookies and writes them to `token_blacklist`. Always succeeds even if the access token is already expired — cookies are cleared regardless.

Does **not** require a valid access token (handles the expired-but-wants-to-logout case).

Response `200`

```
{ "ok": true }
```

POST /api/v1/auth/forgot-password

Initiate a password reset. Returns a reset token in the response body.

Production note: In a deployed environment, replace the `reset_token` field in the response with an email to the user. The token is only returned directly for development convenience.

Request

```
{ "email": "user@example.com" }
```

Response `200` — always 200, even if the email is not registered (prevents user enumeration).

```
{ "message": "If the email is registered, a reset link has been sent." }
```

The reset link is delivered to the user's email address. In dev mode (`EMAIL_ENABLED=false`) the link is logged to backend stdout instead of sent. The underlying token is valid for `PASSWORD_RESET_TOKEN_EXPIRE_MINUTES` (default 60 minutes) and can only be used once.

POST /api/v1/auth/reset-password

Set a new password using a reset token.

Request

```
{
  "token": "Xk9mR3...",
  "new_password": "correct-horse-battery-staple"
}
```

`new_password` must be at least 8 characters.

Response 200

```
{ "message": "Password updated. Please log in with your new password." }
```

Error 400 — token invalid, expired, or already used.

GET /api/v1/users/me

Returns the current authenticated user. Requires a valid access cookie or Bearer token.

Response — `UserResponse` shape (same as the `user` object in login).

RBAC Matrix

Four roles exist. The internal ORM enum value for `RESOURCE_ADMIN` is `ADMIN`; the API always serializes and accepts it as `RESOURCE_ADMIN`.

Capability	MEMBER	GROUP_LEAD	RESOURCE_ADMIN	SUPER_ADMIN
Log in	✓	✓	✓	✓
View own profile (<code>/users/me</code>)	✓	✓	✓	✓
List compute systems	✓	✓	✓	✓
Create bookings	✓	✓	✓	✓
View own bookings	✓	✓	✓	✓
Cancel own booking	✓	✓	✓	✓
View group bookings	—	own group	✓	✓
View group members	—	own group	✓	✓
View group usage	—	own group	✓	✓
Cancel any booking	—	—	✓	✓
Register compute systems	—	—	✓	✓
Manage groups (create/update)	—	—	✓	✓
Manage users (create/update)	—	—	✓	✓
View audit log	—	—	✓	✓
Assign / change roles	—	—	—	✓
Modify admin users	—	—	—	✓

FastAPI dependency guards

```
# In iam/dependencies.py
require_admin      = require_role([UserRole.ADMIN, UserRole.SUPER_ADMIN])
require_super_admin = require_role([UserRole.SUPER_ADMIN])
own_group_or_admin = own_group_or_admin() # factory; used on group endpoints
```

`own_group_or_admin()` passes if the caller is `RESOURCE_ADMIN` or `SUPER_ADMIN`, or if the caller is a `GROUP_LEAD` whose `group_id` matches the `group_id` path parameter.

Login Rate Limiting

Failed login attempts are tracked per identifier (username or email). Successful login clears the counter.

Parameter	Value
Window	10 minutes (fixed)
Max failures before logout	5
Lockout response	HTTP 429 with message "Too many failed login attempts. Try again in 10 minutes."

Two backend implementations share the same interface — resolved once at process startup:

Backend	Active when
<code>_RedisRateLimiter</code>	<code>REDIS_URL</code> is configured and Redis is reachable
<code>_InMemoryRateLimiter</code>	No <code>REDIS_URL</code> set, or Redis is unreachable at startup

The Redis backend uses a Lua `INCR + EXPIRE` script for atomic fixed-window counting; counters survive restarts and work correctly across multiple API replicas. The in-memory fallback resets on process restart and is single-replica only.

If Redis is configured but unreachable at startup, the process falls back to in-memory and logs a warning — it does not crash.

Token Blacklist

Revoked JTIs are stored in the `token_blacklist` table. The table schema is:

```
jti          VARCHAR PRIMARY KEY
expires_at   DATETIME NOT NULL
```

An index on `expires_at` supports efficient cleanup.

When tokens are blacklisted

Event	Tokens blacklisted
POST <code>/auth/logout</code>	Access token + refresh token (both from cookies)
POST <code>/auth/refresh</code>	Old refresh token (rotation — prevents replay)

Cleanup

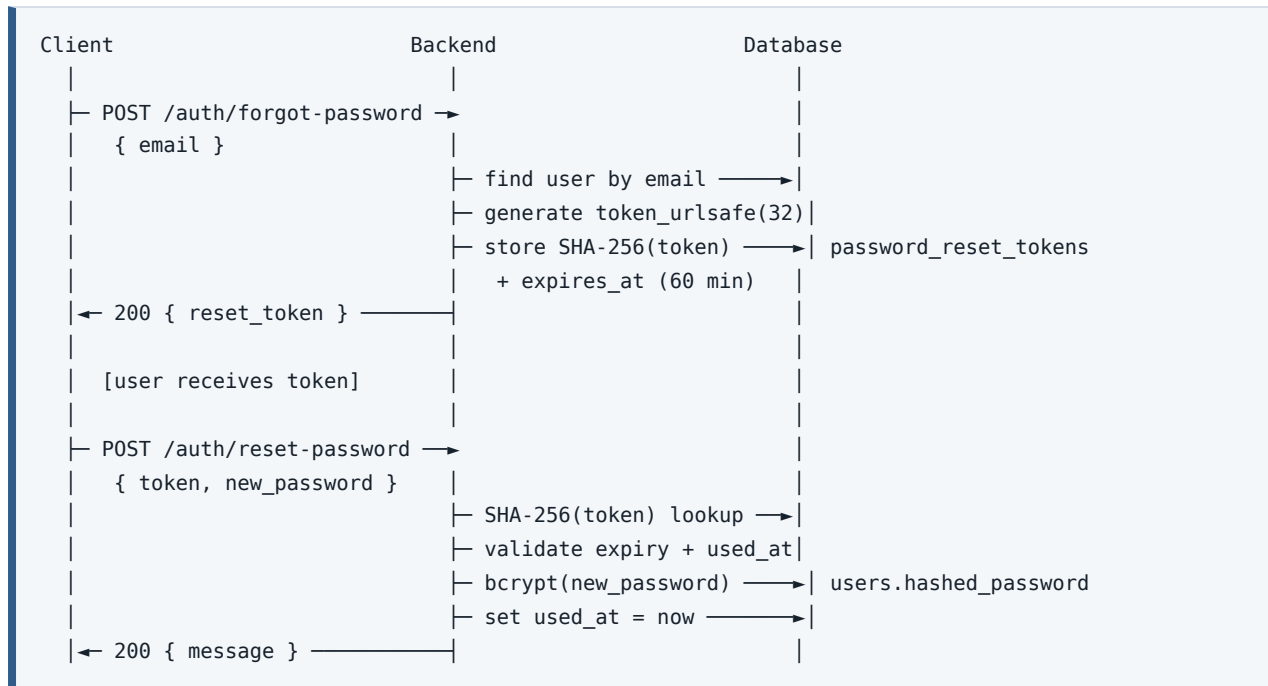
Expired entries are deleted during every run of `reconcile_bookings` (the scheduled cron task). Since an expired token is already rejected by JWT signature verification, stale blacklist rows are harmless but are pruned to keep the table

small.

Manual cleanup:

```
from crags.modules.iam.service import cleanup_expired_blacklist
cleanup_expired_blacklist(db)
```

Password Reset Flow



Security properties: - The database stores only the SHA-256 hash of the token, never the raw value. - Tokens are single-use (`used_at` set on consumption). - Tokens expire after 60 minutes (configurable via `PASSWORD_RESET_TOKEN_EXPIRE_MINUTES`). - The `forgot-password` endpoint always returns `200` regardless of whether the email exists.

Audit Trail

Every auth event writes a row to `audit_logs`:

action	Trigger	user_id
<code>AUTH_LOGIN</code>	Successful login	Authenticated user
<code>AUTH_FAILURE</code>	Wrong password	User ID if account exists, <code>null</code> if not
<code>AUTH_LOGOUT</code>	<code>POST /auth/logout</code> with a valid access token	User from token
<code>AUTH_REFRESH</code>	Successful <code>POST /auth/refresh</code>	User from refresh token
<code>AUTH_RESET_REQUESTED</code>	<code>POST /auth/forgot-password</code> for a known email	User ID
<code>AUTH_RESET_COMPLETED</code>	Successful <code>POST /auth/reset-password</code>	User ID

Audit failures are swallowed (logged but not surfaced to the caller) so an audit write error never blocks authentication.

Auth events use `table_name = "auth"` and `record_id = user_id`, consistent with the existing `bookings` audit format.

Frontend Integration

Auto-refresh interceptor (`api/client.ts`)

The Axios client has a response interceptor that handles `401` responses transparently:

1. Catches any `401` that is not from `/auth/login`, `/auth/refresh`, or `/auth/logout`.
2. If a refresh is already in flight, queues the failed request.
3. Calls `POST /auth/refresh`. If successful, retries all queued requests.
4. If refresh fails (refresh token expired or revoked), dispatches a `crag-session-expired` DOM event and rejects all queued requests.

This means API callers never need to handle token expiry themselves — a failed request is retried once automatically.

Session expiry hook (`hooks/useSessionExpiry.ts`)

```
const { secondsRemaining, isExpiringSoon } = useSessionExpiry(
  loginResponse.access_token_expires_at,
  () => { /* handle forced logout */ }
)
```

Property	Description
<code>secondsRemaining</code>	Seconds until the access token expires (updates every 10 s)
<code>isExpiringSoon</code>	<code>true</code> when ≤ 2 minutes remain — use to show a warning banner

The hook pre-emptively calls `refreshToken()` when ≤ 60 seconds remain, before the interceptor would need to rescue a failed request.

Forced logout (`hooks/useLogoutMutation.ts`)

The logout mutation listens for `crag-session-expired` events (dispatched by the interceptor on refresh failure) and clears the React Query cache, which causes the app to return to the login screen.

Password reset

```
// Request a reset token
const result = await forgotPassword({ email: "user@example.com" })

// Apply the new password
await resetPassword({ token: result.reset_token, new_password: "new-pass-123" })
```

Both functions are in `api/cragApi.ts`. Wire them to form components as needed.

Configuration Reference

All settings are read from environment variables (and `.env`).

Variable	Default	Description
<code>JWT_SECRET_KEY</code>	<code>dev-secret</code>	HMAC secret for signing tokens. Change before any non-local deployment.

Variable	Default	Description
JWT_ALGORITHM	HS256	JWT signing algorithm.
ACCESS_TOKEN_EXPIRE_MINUTES	15	Access token lifetime in minutes.
REFRESH_TOKEN_EXPIRE_DAYS	7	Refresh token lifetime in days.
REFRESH_COOKIE_NAME	crag_refresh	Name of the refresh cookie.
PASSWORD_RESET_TOKEN_EXPIRE_MINUTES	60	Reset token validity window.
AUTH_COOKIE_NAME	crag_session	Name of the access token cookie.
AUTH_COOKIE_SECURE	false	Set <code>true</code> behind HTTPS.
AUTH_COOKIE_SAMESITE	lax	Cookie SameSite policy.
AUTH_COOKIE_DOMAIN	unset	Optional cookie domain scope.

Database Schema

token_blacklist

```
CREATE TABLE token_blacklist (
  jti          VARCHAR PRIMARY KEY,
  expires_at   DATETIME NOT NULL
);
CREATE INDEX ix_token_blacklist_expires_at ON token_blacklist (expires_at);
```

password_reset_tokens

```
CREATE TABLE password_reset_tokens (
  id           INTEGER PRIMARY KEY,
  user_id      INTEGER NOT NULL REFERENCES users(id),
  token_hash   VARCHAR NOT NULL UNIQUE,
  expires_at   DATETIME NOT NULL,
  used_at      DATETIME
);
CREATE INDEX ix_password_reset_tokens_user_id ON password_reset_tokens (user_id);
```

Both tables are created by migration `c7d8e9f0a1b2_add_auth_service_tables`.

Operations

Apply the new migrations

```
# Docker Compose (automatic on next up)
docker-compose up --build

# Local backend-only
cd backend
uv run alembic upgrade head
```

Two new migrations will run in order: 1. `b3c4d5e6f7a8` — makes booking resource columns NOT NULL 2. `c7d8e9f0a1b2` — adds `token_blacklist` and `password_reset_tokens`

Rotate the JWT secret

Rotating `JWT_SECRET_KEY` immediately invalidates all issued tokens (they fail signature verification). Users will be logged out and must re-authenticate.

1. Update `JWT_SECRET_KEY` in `.env`.
2. Restart the backend: `docker-compose restart backend`.
3. Optionally truncate `token_blacklist` since all old JTIs are now unreachable anyway.

Inspect active blacklist entries

```
SELECT jti, expires_at FROM token_blacklist
WHERE expires_at > NOW()
ORDER BY expires_at;
```

Manually expire a user's sessions

Blacklisting is JTI-based, not user-based. To force a specific user out, either:

- Rotate `JWT_SECRET_KEY` (logs out everyone), or
- Set `users.is_active = false` for the target user — `get_current_user` rejects inactive users on every request even if the token is valid.

```
UPDATE users SET is_active = false WHERE username = 'target-user';
```

Re-enable with:

```
UPDATE users SET is_active = true WHERE username = 'target-user';
```

Force a full session cleanup

```
# Wipe the blacklist table (safe to do when JWT_SECRET_KEY has been rotated)
docker-compose exec postgres psql -U crags -d crags -c "TRUNCATE token_blacklist;"

# Expire all unused password reset tokens immediately
docker-compose exec postgres psql -U crags -d crags \
  -c "UPDATE password_reset_tokens SET expires_at = NOW() WHERE used_at IS NULL;"
```

Resources Service

Resources Service

This document covers the compute system registry: data model, API endpoints, RBAC, status enforcement, audit trail, and database schema.

Table of Contents

1. Overview
2. Data Model
3. API Endpoints

- 4. RBAC
- 5. SystemStatus Enforcement
- 6. Capacity Guard on Update
- 7. Audit Trail
- 8. Database Schema

Overview

The Resources service manages the inventory of compute systems that users can book. It tracks hardware capacity, availability status, and exposes CRUD endpoints for administrators. All booking requests validate that the target system is `ACTIVE` before proceeding.

Data Model

SystemType

```
class SystemType(enum.Enum):
    CPU      = "CPU"
    GPU      = "GPU"
    HYBRID   = "HYBRID"
```

SystemStatus

```
class SystemStatus(enum.Enum):
    ACTIVE      = "ACTIVE"      # accepts new bookings
    MAINTENANCE = "MAINTENANCE" # temporarily unavailable, blocked from new bookings
    OFFLINE     = "OFFLINE"     # decommissioned (soft-deleted), blocked from new bookings
```

ComputeSystem

Column	Type	Notes
<code>id</code>	INTEGER PK	Auto-increment
<code>name</code>	VARCHAR UNIQUE	Human-readable identifier
<code>system_type</code>	ENUM(SystemType)	<code>CPU</code> , <code>GPU</code> , or <code>HYBRID</code>
<code>cpu_cores</code>	INTEGER	Total CPU cores
<code>ram_gb</code>	INTEGER	Total RAM in GB
<code>gpu_units</code>	INTEGER	Total GPU count
<code>vram_gb</code>	INTEGER	Total VRAM in GB
<code>status</code>	ENUM(SystemStatus)	Defaults to <code>ACTIVE</code>
<code>health_check_url</code>	TEXT (nullable)	HTTP endpoint pinged by the health monitor on every reconciler tick
<code>last_health_check_at</code>	DATETIME (nullable)	Timestamp of the most recent health check
<code>last_health_status</code>	VARCHAR(20) (nullable)	<code>UP</code> , <code>DOWN</code> , or <code>SKIP</code>

API Endpoints

All endpoints are under `/api/v1/systems/`.

GET `/api/v1/systems/`

List all registered systems. Any authenticated user may call this.

Caching: The unfiltered response (`status` query param omitted) is cached for **60 seconds** using the application-level Redis or in-memory cache. Any write to a system (POST, PATCH, DELETE) or a health-check status transition busts the cache immediately. Filtered queries (`?status=ACTIVE` etc.) always hit the database directly.

Query parameters

Parameter	Type	Description
<code>status</code>	<code>ACTIVE</code> <code>MAINTENANCE</code> <code>OFFLINE</code>	Filter by status. Omit to return all.

Response `200`

```
[
  {
    "id": 1,
    "name": "gpu-cluster-01",
    "system_type": "GPU",
    "cpu_cores": 64,
    "ram_gb": 512,
    "gpu_units": 8,
    "vram_gb": 640,
    "status": "ACTIVE"
  }
]
```

GET `/api/v1/systems/{system_id}`

Fetch a single system by ID. Any authenticated user may call this.

Response `200` — same shape as a single element from the list endpoint.

Error `404` — system not found.

POST `/api/v1/systems/`

Register a new compute system. Requires `RESOURCE_ADMIN` or `SUPER_ADMIN`.

Request

```
{
  "name": "gpu-cluster-02",
  "system_type": "GPU",
  "cpu_cores": 32,
  "ram_gb": 256,
  "gpu_units": 4,
  "vram_gb": 320,
  "status": "ACTIVE"
}
```

`status` is optional and defaults to `ACTIVE`.

`system_type` must be one of `CPU`, `GPU`, `HYBRID`. Invalid values return `422`.

Response `200` — the created `SystemResponse`.

Error `422` — validation failure (invalid enum value, negative capacity).

PATCH `/api/v1/systems/{system_id}`

Partially update a system. All fields are optional. Requires `RESOURCE_ADMIN` or `SUPER_ADMIN`.

Request — include only fields to change.

```
{
  "status": "MAINTENANCE",
  "gpu_units": 6
}
```

Response `200` — updated `SystemResponse`.

Error `404` — system not found.

Error `409` — capacity reduction rejected because active bookings require more resources. See [Capacity Guard on Update](#).

DELETE `/api/v1/systems/{system_id}`

Soft-delete a system by setting its status to `OFFLINE`. The system record and all its booking history are retained. Requires `RESOURCE_ADMIN` or `SUPER_ADMIN`.

This is a soft delete, not a hard row delete. Booking history and foreign keys are preserved.

Response `200` — the system with `"status": "OFFLINE"`.

Error `404` — system not found.

RBAC

Capability	MEMBER	GROUP_LEAD	RESOURCE_ADMIN	SUPER_ADMIN
List systems	✓	✓	✓	✓
Get system by ID	✓	✓	✓	✓
Register system	—	—	✓	✓
Update system	—	—	✓	✓

Capability	MEMBER	GROUP_LEAD	RESOURCE_ADMIN	SUPER_ADMIN
Decommission system	—	—	✓	✓

SystemStatus Enforcement

When a booking is created, `scheduling/service.py` checks the target system's status **before** any capacity math:

```
if system.status != SystemStatus.ACTIVE:
    raise ValueError(
        f"System '{system.name}' is not available for booking (status:
{system.status.value})"
    )
```

This raises `HTTP 409` at the booking endpoint. Setting a system to `MAINTENANCE` or `OFFLINE` immediately blocks all new bookings against it. Existing confirmed bookings are unaffected — cancel them separately if needed.

Capacity Guard on Update

`PATCH /systems/{id}` rejects any capacity reduction that would leave active bookings (`REQUESTED` or `CONFIRMED`) under-resourced:

```
Cannot reduce gpu_units to 2 – active bookings require at least 4
```

The check uses the maximum of each resource field across all active bookings on that system. Reducing capacity is permitted when no active bookings exist, or when the new value is still $\geq$ the maximum actively booked value.

The update query uses `SELECT ... FOR UPDATE` to prevent a race between a concurrent booking creation and a capacity change.

Audit Trail

Every mutating operation writes a row to `audit_logs`:

action	Trigger
<code>SYSTEM_CREATED</code>	<code>POST /systems/</code>
<code>SYSTEM_UPDATED</code>	<code>PATCH /systems/{id}</code>
<code>SYSTEM_DELETED</code>	<code>DELETE /systems/{id}</code> (soft-delete)

```
table_name = "compute_systems", record_id = system.id, user_id = actor.id.
```

Database Schema

```
CREATE TYPE systemtype AS ENUM ('CPU', 'GPU', 'HYBRID');
CREATE TYPE systemstatus AS ENUM ('ACTIVE', 'MAINTENANCE', 'OFFLINE');

CREATE TABLE compute_systems (
  id INTEGER PRIMARY KEY,
  name VARCHAR UNIQUE,
  system_type systemtype,
  cpu_cores INTEGER,
  ram_gb INTEGER,
  gpu_units INTEGER,
  vram_gb INTEGER,
  status systemstatus,
  health_check_url TEXT,
  last_health_check_at TIMESTAMPTZ,
  last_health_status VARCHAR(20)
);
```

The enum types are native PostgreSQL enums created by migration `25c2237342f5`. Health columns were added by migration `d7e8f9a0b1c2`.

Notifications And Webhooks

Notifications Service

This document covers the email notification layer: delivery mechanism, templates, trigger points, configuration, and extension guide.

Table of Contents

1. Overview
2. Architecture
3. Email Templates
4. Trigger Points
5. Password Reset Flow Change
6. Configuration Reference
7. Dev vs Production Mode
8. SMTP Compatibility
9. Adding a New Email
10. Module Layout

Overview

The Notifications service sends plain-text emails for three events: password reset, booking confirmed, and booking preempted. It uses Python's built-in `smtpplib` — no third-party email library or paid service is required.

Email delivery is **fire-and-forget**: a failed send is logged but never raises to the caller and never rolls back a database transaction.

Architecture

```

Event occurs in a service layer
    |
    ▼
notifications/service.py :: send_email(to, subject, body)
    |
    ├── EMAIL_ENABLED=False → log to stdout (dev / CI)
    |
    └── EMAIL_ENABLED=True → smtplib.SMTP → SMTP relay
                                |
                                ├── STARTTLS (SMTP_USE_TLS=True)
                                ├── login() (if SMTP_USER set)
                                └── send_message()

```

`smtplib` is part of the Python standard library. **No additional packages are installed.**

Email Templates

All templates live in `notifications/templates.py`. Each returns `(subject: str, body: str)`. Content is plain text — no HTML, no templating engine.

`password_reset_email(reset_url)`

Triggered by `POST /api/v1/auth/forgot-password`.

```

Subject: CRAGS – Password reset request

You requested a password reset for your CRAGS account.

Click the link below to set a new password. The link expires in 60 minutes
and can only be used once.

https://crag.example.com/reset-password?token=Xk9mR3...

If you did not request this reset, you can safely ignore this email.

```

`reset_url` is constructed as `{FRONTEND_URL}/reset-password?token={raw_token}`.

`booking_confirmed_email(...)`

Triggered immediately after a booking is committed to the database.

```

Subject: CRAGS – Booking #42 confirmed on gpu-cluster-01

Hi alice,

Your resource booking has been confirmed.

System   : gpu-cluster-01
Window   : 2026-06-01T09:00:00+00:00 → 2026-06-01T17:00:00+00:00
CPU       : 8 cores
GPU       : 2 units
RAM       : 64 GB
VRAM      : 80 GB
Booking   : #42

```

Parameters: `username`, `system_name`, `start_time`, `end_time`, `req_cpu`, `req_gpu`, `req_ram`, `req_vram`, `booking_id`.

`booking_preempted_email(...)`

Triggered for each background booking that is preempted to free capacity for a foreground job.

```
Subject: CRAGS – Booking #37 preempted on gpu-cluster-01

Hi bob,

Your background booking has been preempted to free capacity for a foreground job.

System   : gpu-cluster-01
Window   : 2026-06-01T10:00:00+00:00 → 2026-06-01T14:00:00+00:00
Booking  : #37

You can submit a new booking request in the CRAGS portal at any time.
```

Parameters: `username`, `system_name`, `start_time`, `end_time`, `booking_id`.

Trigger Points

Event	File	Condition
Password reset requested	<code>iam/router.py</code> → <code>forgot_password()</code>	User exists and has an email address
Booking confirmed	<code>scheduling/service.py</code> → <code>create_booking()</code>	After <code>db.commit()</code> ; actor user has an email
Booking preempted	<code>scheduling/service.py</code> → <code>preempt_background_jobs()</code>	Per preempted job; owner has an email

Email is only sent when the user has a non-empty `email` field. If `email` is `None` or empty, the send is skipped silently.

Password Reset Flow Change

Prior to this service, `POST /auth/forgot-password` returned the raw reset token in the response body — a development shortcut that would leak the token in production. That field is now removed.

Old response:

```
{
  "message": "If the email is registered, a reset token has been issued.",
  "reset_token": "Xk9mR3..."
}
```

New response:

```
{
  "message": "If the email is registered, a reset link has been sent."
}
```

The reset link is delivered only via email. In dev mode (`EMAIL_ENABLED=False`) the link is logged to the backend's stdout so you can still test the flow without an SMTP server.

Configuration Reference

All variables are read from environment / `.env` .

Variable	Default	Description
<code>EMAIL_ENABLED</code>	<code>false</code>	<code>false</code> = log only, no SMTP connection opened
<code>FRONTEND_URL</code>	<code>http://localhost:5173</code>	Base URL prepended to password reset links
<code>EMAIL_FROM</code>	<code>crag@localhost</code>	<code>From:</code> header on all outgoing mail
<code>SMTP_HOST</code>	<code>localhost</code>	SMTP server hostname
<code>SMTP_PORT</code>	<code>587</code>	SMTP port (587 = STARTTLS submission, 465 = implicit TLS, 25 = plain)
<code>SMTP_USER</code>	<i>(empty)</i>	SMTP login username — leave empty to skip auth
<code>SMTP_PASSWORD</code>	<i>(empty)</i>	SMTP login password
<code>SMTP_USE_TLS</code>	<code>true</code>	Call <code>STARTTLS</code> before authenticating

Example `.env` for production with an SMTP relay

```
EMAIL_ENABLED=true
FRONTEND_URL=https://crag.example.com
EMAIL_FROM=noreply@crag.example.com
SMTP_HOST=smtp.example.com
SMTP_PORT=587
SMTP_USER=noreply@crag.example.com
SMTP_PASSWORD=supersecret
SMTP_USE_TLS=true
```

Example `.env` for local dev (Mailpit / MailHog)

```
EMAIL_ENABLED=true
FRONTEND_URL=http://localhost:5173
EMAIL_FROM=crag@localhost
SMTP_HOST=localhost
SMTP_PORT=1025
SMTP_USER=
SMTP_PASSWORD=
SMTP_USE_TLS=false
```

[Mailpit](#) and [MailHog](#) are free, open-source local SMTP catchers with a web UI — ideal for testing email in development without sending real mail.

Dev vs Production Mode

Setting	Behaviour
<code>EMAIL_ENABLED=false</code> (default)	<code>send_email()</code> logs the full email (To, Subject, body) at <code>INFO</code> level and returns. No socket is opened.
<code>EMAIL_ENABLED=true</code>	Connects to <code>SMTP_HOST:SMTP_PORT</code> , optionally calls <code>STARTTLS</code> and <code>login()</code> , then sends. Any exception is caught and logged at <code>ERROR</code> level — the caller never sees it.

SMTP Compatibility

`smtplib` works with any standards-compliant SMTP server or relay:

Provider	Notes
Postfix / Sendmail	Port 25 or 587, <code>SMTP_USE_TLS</code> as needed
Mailpit / MailHog (dev)	Port 1025, <code>SMTP_USE_TLS=false</code>
Gmail SMTP	Port 587, <code>SMTP_USE_TLS=true</code> , app password required
Brevo (Sendinblue)	Port 587, free tier available
Mailgun SMTP	Port 587, free tier available
AWS SES SMTP	Port 587, IAM credentials as user/password

All of the above are free to self-host or have a free tier. None require a paid SDK or proprietary library.

Adding a New Email

1. Add a template in `notifications/templates.py`:

```
def my_event_email(*, param1: str, param2: int) -> tuple[str, str]:
    subject = "CRAGS – My event"
    body = f"..."
    return subject, body
```

2. Call `send_email` at the relevant point in the service layer:

```
from crags.modules.notifications.service import send_email
from crags.modules.notifications.templates import my_event_email

if user.email:
    send_email(user.email, *my_event_email(param1=..., param2=...))
```

No registration, no config change, no router change required.

Module Layout

```
backend/src/crags/modules/notifications/
├── __init__.py
├── service.py      send_email() – SMTP delivery or dev-mode logging
└── templates.py   password_reset_email, booking_confirmed_email, booking_preempted_email
```


Scheduling Engine

Scheduler

This document covers the background job scheduler: what it does, how it works, configuration, concurrency model, and the scaling roadmap from a single-process deployment to a distributed task queue.

Table of Contents

1. Overview
2. Jobs
3. Reconcile Algorithm
4. Concurrency and Locking
5. Lifecycle
6. Health Check
7. Configuration Reference
8. Scaling Roadmap
9. Future Algorithm Improvements
10. Operations

Overview

CRAGS uses `APScheduler` (`BackgroundScheduler`) to run periodic maintenance jobs inside the FastAPI process. APScheduler is MIT-licensed; no paid service or external broker is required for the current single-process deployment.

The scheduler starts on application startup, runs a reconciliation job immediately, and then repeats on a configurable interval. It shuts down cleanly when the application exits.

Jobs

`reconcile_bookings`

Property	Value
Trigger	<code>interval</code>
Default interval	5 minutes (<code>RECONCILE_INTERVAL_MINUTES</code>)
First run	Immediately on startup (<code>next_run_time = now</code>)
Max concurrent instances	1 (<code>max_instances=1</code>)
Missed fire behaviour	Coalesce — skip missed fires, run once when back
Misfire grace period	60 seconds

Reconcile Algorithm

Each invocation of `_run_reconcile()` in `main.py` performs seven steps in order. Each step uses its own database session opened and closed independently to prevent state leakage.

```
Step 1: CONFIRMED + upper(booking_period) < now → COMPLETED
Step 2: REQUESTED + upper(booking_period) < now → EXPIRED
Step 3: DELETE FROM token_blacklist WHERE expires_at < now
Step 4: DELETE FROM audit_logs WHERE timestamp < now - AUDIT_RETAIN_DAYS
Step 5: maintenance.service.apply_maintenance_transitions
Step 6: waitlist.service.promote_eligible
Step 7: health.service.run_health_checks
```

Step 1 — COMPLETED

Bookings that were `CONFIRMED` and whose time window has fully elapsed are marked `COMPLETED`. This is the normal terminal state for a booking that ran to completion.

Step 2 — EXPIRED

Bookings that were `REQUESTED` (pending admin approval) and whose time window has fully elapsed are marked `EXPIRED`. A requested booking can only be confirmed while its window is in the future.

Step 3 — Token blacklist pruning

Expired JWT entries in `token_blacklist` are deleted. They are already rejected by JWT signature verification, so they are harmless; pruning keeps the table small.

Step 4 — Audit log retention

Audit rows older than `AUDIT_RETAIN_DAYS` (default 90) are deleted. This bounds unbounded growth of the audit table.

Step 5 — Maintenance window transitions

`apply_maintenance_transitions` scans `maintenance_windows`: - Windows whose `start_time ≤ now` and that haven't been applied yet: set the target system to `MAINTENANCE` and cancel any `CONFIRMED` or `REQUESTED` bookings that overlap the window. - Windows whose `end_time ≤ now` and that are still in effect: restore the system to `ACTIVE` (unless the health monitor reports it as `DOWN`).

Step 6 — Waitlist promotion

`promote_eligible` scans `WAITING` waitlist entries ordered by priority score (`GROUP_LEAD/ADMIN` get +10) then creation time. For each entry, it checks whether the requested resources fit the system's current capacity in a window starting from `now` for `duration_hours`. If capacity exists, it creates a `CONFIRMED` booking and marks the entry `PROMOTED`.

Step 7 — Health checks

`run_health_checks` pings each system's `health_check_url` (if set) with a 5-second timeout: - `UP` (HTTP 2xx): if the system is in `MAINTENANCE` and no scheduled maintenance window is active, restore it to `ACTIVE`. - `DOWN` (non-2xx or error): if the system is `ACTIVE`, set it to `MAINTENANCE`. - `SKIP`: no `health_check_url` configured; system status unchanged.

After any status changes, the `craggs:cache:systems` response cache is invalidated.

Return value

`reconcile_bookings(db)` (Step 1–4) returns a summary dict logged at `INFO` level:

```
{
    "completed": 3,
    "expired": 1,
    "blacklist_pruned": 12,
    "audit_pruned": 0,
}
```

Concurrency and Locking

SELECT ... FOR UPDATE SKIP LOCKED

Both booking queries (Steps 1 and 2) use `with_for_update(skip_locked=True)` :

```
db.query(Booking)
    .filter(...)
    .with_for_update(skip_locked=True)
    .all()
```

Why `FOR UPDATE` : Prevents a concurrent `create_booking` transaction from reading a row that the reconciler is mid-way through updating, or vice versa.

Why `SKIP LOCKED` : If two scheduler instances fire at the same time (e.g. two backend replicas both running `BackgroundScheduler`), the second instance skips any rows already locked by the first rather than blocking. This makes the reconciler safe to run concurrently across replicas without deadlocks or duplicate state transitions.

Idempotency: Every state transition is idempotent. Running the same reconciliation twice on the same row produces the same result — the second pass just finds no matching rows.

Session isolation

`_run_reconcile()` in `main.py` opens a dedicated `SessionLocal()` session for each job execution and closes it in a `finally` block. This session is independent from the request-scoped sessions used by API endpoints.

Lifecycle

```
Application starts
|
├─ _check_security_settings()
├─ ensure_super_admin(db)
├─ _scheduler.add_job("reconcile_bookings", next_run_time=now)
├─ _scheduler.start()
|
|   └─ Fires immediately (next_run_time=now)
|   └─ Fires every RECONCILE_INTERVAL_MINUTES thereafter
|   └─ ...
|
Application shuts down (SIGTERM / SIGINT)
├─ _scheduler.shutdown(wait=False)
|   └─ Running job completes; no new jobs start
```

`wait=False` on shutdown means the application does not block waiting for a job to finish. A running reconciliation will be interrupted at the next DB commit boundary. Because all operations are idempotent, the next startup will safely re-process any rows that were not committed.

Health Check

`GET /healthz` now exposes scheduler state:

```
{
  "status": "ok",
  "scheduler": {
    "running": true,
    "next_reconcile": "2026-05-26T12:05:00+00:00"
  }
}
```

Field	Description
<code>scheduler.running</code>	<code>true</code> if the APScheduler instance is active
<code>scheduler.next_reconcile</code>	ISO timestamp of the next scheduled fire; <code>null</code> if the scheduler is not running

Configuration Reference

Variable	Default	Description
<code>RECONCILE_INTERVAL_MINUTES</code>	<code>5</code>	How often <code>reconcile_bookings</code> runs, in minutes
<code>AUDIT_RETAIN_DAYS</code>	<code>90</code>	Audit log rows older than this are deleted on each reconcile run

Scaling Roadmap

Phase 1 — Current: `BackgroundScheduler` (in-process)

One scheduler instance per process. Correct for single-replica deployments. With `SKIP_LOCKED`, two replicas firing simultaneously are safe but wasteful (N-1 extra runs per tick).

```
[Uvicorn process]
└─ BackgroundScheduler thread
    └─ reconcile_bookings() every 5 min
```

Phase 2 — Multi-replica: `SQLAlchemyJobStore`

APScheduler persists job state in the shared database. Only the replica that wins the advisory lock for a given fire time executes the job; others skip it. No new infrastructure required — just a `pyproject.toml` change and scheduler init change.

```
from apscheduler.jobstores.sqlalchemy import SQLAlchemyJobStore

_scheduler = BackgroundScheduler(
    jobstores={"default": SQLAlchemyJobStore(url=settings.DATABASE_URL)},
    job_defaults={"coalesce": True, "max_instances": 1},
)
```

Install: `uv add "apscheduler[sqlalchemy]"` (still MIT, no paid tier).

Phase 3 — High-scale: Celery + Redis Beat

Separates scheduling (Celery Beat) from execution (Celery Workers). Recommended when you need:

- Dozens of replicas

- Task retries with exponential back-off
- Dead-letter queue for failed jobs
- Task execution history

```

Redis (broker + Beat store)
|
Celery Beat → fires task every N min
|
Celery Workers (pool) → one worker picks up the task
|
PostgreSQL (CRAGS DB)

```

All components are free and open-source (Celery: MIT; Redis: BSD; redis-py: MIT).

The `reconcile_bookings` function itself does not change across any of these phases — only the scheduler init and the infrastructure change.

Future Algorithm Improvements

Current limitation	Improvement
<code>REQUESTED → EXPIRED</code> transition not audited	Write <code>BOOKING_EXPIRED</code> audit row per expired booking in Step 2
Preemption selects by GPU descending then ID	Priority-weighted score: <code>score = req_gpu / priority_tier</code> — lower-priority jobs preempted first
No back-fill after preemption	After freeing capacity, scan <code>REQUESTED</code> bookings from the same group and auto-confirm those that now fit
Monthly quota re-scans all bookings per request	Replace Python-side <code>sum()</code> with a single <code>SELECT SUM(duration * req_cpu) GROUP BY group_id, month</code> aggregate query
<code>booking_period</code> scans all rows to find expired	Partial index on <code>(status, upper(booking_period))</code> — reconcile only touches relevant rows

Operations

Trigger a manual reconcile

```

# From a Python shell with the DB reachable:
from crags.db.session import SessionLocal
from crags.modules.scheduling.cron import reconcile_bookings

db = SessionLocal()
result = reconcile_bookings(db)
db.close()
print(result)

```

Change the interval without restart

```
from crags.main import _scheduler
_scheduler.reschedule_job("reconcile_bookings", trigger="interval", minutes=1)
```

Pause / resume the scheduler

```
_scheduler.pause() # stops firing jobs; does not affect running ones
_scheduler.resume() # re-enables firing
```

Check job status

```
curl http://localhost:8000/healthz
```

User Manual

CRAGS User Manual

Compute Resource Allocation and Governance System

This guide explains how to use CRAGS to request and manage compute resource bookings.

Table of Contents

1. Getting Started
2. Logging In
3. Understanding the Dashboard
4. Viewing Compute Systems
5. Creating a Booking Request
6. Managing Your Bookings
7. Understanding Booking Status
8. Audit Trail
9. Team Management (Group Leads)
10. Resource Quotas and Limits
11. Troubleshooting

Getting Started

CRAGS is accessible from any web browser. Contact your system administrator for:

- The CRAGS application URL
- Your login credentials (username, email, or password reset link)
- Any proxy or network configuration needed

Browser Support: Chrome, Firefox, Safari, Edge (modern versions recommended)

Logging In

1. Open the CRAGS application URL in your browser
2. Click the **Login** button
3. Enter your credentials:

- **Identifier:** Your username or email
- **Password:** Your password

4. Click **Login**

If you forget your password, contact your group administrator or system administrator.

Session Duration

Sessions remain active for 60 minutes of inactivity. You will be logged out automatically and must log back in to continue.

Understanding the Dashboard

After logging in, you arrive at the **Mission Control Dashboard**, which displays:

- **Your Role:** Displays your user role (User, Group Lead, Resource Admin, Super Admin)
 - **Group Membership:** Shows your group name and members
 - **Quick Stats:** Overview of your recent bookings and resource usage
 - **Navigation Tabs:**
 - **Systems:** View and manage compute systems
 - **Bookings:** Request and manage resource bookings
 - **Team** (Group Leads only): Manage group members and quotas
 - **Audit:** View history of all allocation decisions
-

Viewing Compute Systems

The **Systems** tab displays all available compute resources.

System Information

Each system shows:

- **System Name:** Identifier for the resource cluster
- **Type:** CPU, GPU, or HYBRID (mixed resources)
- **Available Resources:**
 - CPU cores
 - GPU units
 - RAM (GB)
 - VRAM (GB)

Filtering Systems

Use the system list to find resources by name or type.

Creating a Booking Request

Step 1: Navigate to Bookings

Click the **Bookings** tab on the dashboard.

Step 2: Click “New Booking”

Click the **New Booking** button to open the booking form.

Step 3: Fill in Booking Details

System Selection - Select the compute system you want to use - The form displays available capacity

Time Window - **Start Time:** When you need the resources (ISO datetime) - **End Time:** When you will release the resources - The system shows available capacity for your chosen time window

Resource Request - CPU Cores: Number of CPU cores needed - **GPU Units:** Number of GPUs needed - **RAM (GB):** Amount of RAM in gigabytes - **VRAM (GB):** Amount of GPU memory needed

Workload Type - Foreground: Interactive, high-priority work - Runs immediately if capacity allows - Can preempt eligible background jobs - Typical use: interactive development, debugging, interactive analysis - **Background:** Batch processing, lower priority - Runs when resources are available - Can be preempted by foreground work - Typical use: batch training, long-running experiments, data processing

Project Information - Academic Category: Research, Teaching, Administration, Other - **Project Title:** Brief name of your project - **Expected Deliverable:** What you're producing (paper, model, dataset, etc.) - **Objective:** What you're trying to accomplish with these resources

Step 4: Review and Submit

1. Review your booking request
2. Click **Request Booking**
3. The system will either:
 - **Accept** immediately (status: CONFIRMED) if capacity allows
 - **Reject** with detailed explanation if capacity is insufficient

Important Notes

- **Exact Fit Required:** Your resource request must not exceed available capacity
- **No Partial Allocations:** If GPU units are unavailable, your entire GPU request is rejected
- **Time Windows:** Requests within the exact time window are checked
- **Quota Limits:** Your group's quota limits also affect approval

Managing Your Bookings

Viewing Bookings

The **Bookings** tab shows all your booking requests.

Booking Details

Each booking displays:

- **Status:** Current state (REQUESTED, CONFIRMED, CANCELLED, PREEMPTED, COMPLETED, EXPIRED)
- **System:** Which resource cluster
- **Time Window:** Requested dates and times
- **Resources:** CPU, GPU, RAM, VRAM requested
- **Type:** Foreground or background
- **Project Info:** Academic category and project title

Cancelling a Booking

1. Click the booking you want to cancel
2. Click the **Cancel Booking** button
3. Confirm the cancellation

Cancelled bookings remain visible in your history for audit purposes.

Filtering and Sorting

Use the filter options to find bookings by: - System - Status - Academic category - Date range

Understanding Booking Status

Status	Meaning
REQUESTED	Booking request submitted, awaiting system decision
CONFIRMED	Request approved; your resources are reserved
CANCELLED	You cancelled the booking
PREEMPTED	Your background job was interrupted for a foreground job
COMPLETED	Booking window ended normally
EXPIRED	Booking was never actioned and the window passed

Status Transitions

```

REQUESTED → CONFIRMED (accepted)
      ↓
      CANCELLED (you cancelled)

CONFIRMED → PREEMPTED (only for background jobs)
      ↓
      CANCELLED (you cancelled or admin cancelled)
      ↓
      COMPLETED (booking window ended normally)

```

What “PREEMPTED” Means

If your **background** job is preempted: - A foreground job required your reserved resources - Your job is stopped and the resources released - Your booking status changes to PREEMPTED - You can request the resources again later (Foreground jobs cannot be preempted.)

Audit Trail

The **Audit** tab logs all system decisions and resource usage.

What’s Recorded

- **Booking Creation:** When and who requested
- **Booking Approval/Rejection:** When decision was made and why
- **Resource Usage:** What resources were actually used
- **Cancellations:** When bookings were cancelled
- **Preemptions:** When background jobs were interrupted

Interpreting Audit Entries

Each entry shows:

- **Timestamp:** When the event occurred
- **User:** Who initiated the action
- **Action:** What was done (REQUEST, CONFIRM, CANCEL, etc.)
- **Details:** Relevant information (system, resources, reason)

Why Audit Matters

Audit trails help institutions: - Understand resource usage patterns - Ensure fair allocation across groups - Track research output and impact - Investigate disputes about bookings

Team Management (Group Leads)

If you are a **Group Lead**, you have additional responsibilities.

Access Group Management

Click the **Team** tab to manage your group.

View Group Members

The **Team** tab displays: - List of group members - Their roles (User, Group Lead) - When they joined - Active projects

Manage Group Quotas

Group Leads can view (and Resource Admins can modify) group resource quotas:

Concurrent Quotas (resources in use at the same time) - Concurrent CPU cores limit - Concurrent GPU units limit - Concurrent RAM (GB) limit - Concurrent VRAM (GB) limit

Monthly Quotas (total usage per calendar month) - Monthly CPU core-hours - Monthly GPU core-hours - Monthly RAM GB-hours - Monthly VRAM GB-hours

Group Usage

Click **Usage** to see your group's monthly consumption: - Current month resource usage - Percentage of quota used - Trend compared to previous months

Resource Quotas and Limits

Understanding quotas helps you plan bookings effectively.

How Quotas Work

Concurrent Quotas limit how much your group can use *simultaneously* - Example: If your group's GPU quota is 4, only 4 GPUs can be in use at once - Multiple overlapping bookings count toward the concurrent limit - Non-overlapping bookings do not count toward the concurrent limit

Monthly Quotas limit total usage *per calendar month* - Example: If your group's monthly GPU quota is 500 core-hours, you can use 500 GPU-hours total in the month - Usage is measured in core-hours: (GPU cores used) × (hours booked) - Once quota is exceeded, no new bookings are approved until the next month

Checking Your Quota Status

1. Open the **Team** tab (if you're a Group Lead)
2. View current month usage
3. Compare against your group's quotas
4. Plan future bookings accordingly

What Happens When Quota is Exceeded

If a booking request would exceed your group's quota:

- **Concurrent Quota:** The request is **rejected**
 - Error message: "GROUP_CONCURRENT_QUOTA_EXCEEDED"
 - Solution: Wait for existing bookings to end or request fewer resources
- **Monthly Quota:** The request is **rejected**
 - Error message: "GROUP_MONTHLY_QUOTA_EXCEEDED"
 - Solution: Wait until next month or request fewer resources

Troubleshooting

Booking Request Rejected: "CAPACITY_EXCEEDED"

Your resource request exceeds available capacity.

Solutions: - Reduce your resource request (fewer CPUs, GPUs, RAM, or VRAM) - Choose a different time window when resources are available - View the temporal booking visualization to find free slots - Contact your system administrator if capacity needs increase

Booking Request Rejected: “PREEMPTION_INSUFFICIENT”

You requested foreground resources, but there aren't enough background jobs to preempt.

Solution: - Try requesting fewer resources - Try a different time window - Use background workload type if your work is flexible - Contact your Resource Administrator if you need foreground guarantees

Booking Request Rejected: “GROUP_QUOTA_EXCEEDED”

Your group has reached its quota limit.

Solutions: - Wait until next calendar month for monthly quotas to reset - Contact your Group Lead or Resource Administrator - Request fewer resources or shorter time windows

Session Expired / Need to Log Back In

Solution: Click **Login** and enter your credentials again. Your bookings are preserved.

Cannot See Team Tab

You must be a **Group Lead** to manage team. Contact your group administrator.

Timestamps Display Incorrectly

CRAGS uses ISO 8601 UTC timestamps. Verify: - Your browser timezone settings are correct - You're interpreting times as UTC (or convert to your local timezone)

Still Have Issues?

Contact your **System Administrator** or **Resource Administrator** for: - Password resets - Access issues - Quota adjustments - System capacity questions - Booking disputes

Backend Guide

CRAGS Backend

Compute Resource Allocation and Governance System (CRAGS)

This directory contains the **backend service** for the Compute Resource Allocation and Governance System (CRAGS).

The backend provides the **core scheduling engine, resource management APIs, authentication services, and governance auditing infrastructure** required for managing institutional compute resources.

The service is built using **FastAPI** and **PostgreSQL**, following a modular architecture that separates authentication, resource management, scheduling logic, and audit processing.

1. System Overview

The backend implements the following major responsibilities:

- Management of compute systems and resources
- Scheduling and booking of compute resources
- Priority-aware foreground and background task handling

- User authentication and group management
- Governance auditing of compute resource usage
- Exposure of REST APIs for frontend interaction

The backend follows a **service-oriented modular design**, allowing independent development and evolution of system modules.

2. Technology Stack

Component	Technology
API Framework	FastAPI
Language	Python
Database	PostgreSQL
ORM	SQLAlchemy
Migrations	Alembic
Authentication	JWT-based authentication
Task Scheduling	Python cron / internal scheduler
Containerization	Docker (optional)

3. Directory Structure

```

backend/
|
|─ alembic.ini
|─ pyproject.toml
|─ run.sh
|─ main.py
|
|─ migrations/
|   |─ env.py
|   └─ versions/
|
|─ src/
|   └─ crags/
|       |
|       |─ main.py
|       |
|       |─ api/
|       |   └─ router.py
|       |
|       |─ core/
|       |   |─ config.py
|       |   └─ security.py
|       |
|       |─ db/
|       |   |─ base.py
|       |   |─ session.py
|       |   └─ test_connection.py
|       |
|       └─ modules/
|           |
|           |─ iam/
|           |
|           |─ resources/
|           |
|           |─ scheduling/
|           |
|           └─ audit/

```

4. Core Architecture

The backend is divided into **independent functional modules**.

```

Frontend
|
▼
FastAPI Router Layer
|
▼
Service Layer
|
▼
Database Layer (SQLAlchemy)
|
▼
PostgreSQL

```

Each module provides:

- Database models
- Business logic services
- API routers
- Request/response schemas

5. Core Modules

5.1 IAM (Identity and Access Management)

Directory:

```
src/crag/modules/iam/
```

Responsibilities:

- User registration and authentication
- Password hashing and validation
- Token-based authentication
- Role and group membership management

Key files:

```
models.py  
schemas.py  
router.py  
service.py  
auth_service.py  
dependencies.py
```

5.2 Resources Module

Directory:

```
src/crag/modules/resources/
```

Responsibilities:

- Management of compute systems
- Tracking system capacity
- Registering new compute resources
- Providing system inventory APIs

Example resource types:

- GPU compute nodes
- CPU compute servers
- shared experimental hardware

Key files:

```
models.py  
schemas.py  
router.py  
service.py
```

5.3 Scheduling Module

Directory:

```
src/crags/modules/scheduling/
```

This module implements the **core compute scheduling engine**.

Responsibilities:

- Booking compute resources
- Preventing scheduling conflicts
- Managing foreground and background access
- Handling resource preemption
- Triggering audit events

Key files:

```
models.py  
schemas.py  
router.py  
service.py  
cron.py
```

5.4 Audit Module

Directory:

```
src/crags/modules/audit/
```

Responsibilities:

- Tracking resource usage
- Logging booking activity
- Generating governance reports
- Supporting institutional oversight

Audit data includes:

- compute consumption
- booking history
- preemption events
- weekly usage summaries

6. Database Layer

Database logic is centralized in:

```
src/crags/db/
```

Files:

base.py

Registers all ORM models.

session.py

Defines SQLAlchemy session management.

test_connection.py

Used for validating database connectivity.

7. Database Migrations

The system uses **Alembic** for schema versioning.

Migration scripts are located in:

```
backend/migrations/versions/
```

Example migrations include:

- creation of compute system tables
- creation of bookings table
- creation of IAM tables
- addition of audit infrastructure
- booking period indexing for scheduling efficiency

8. Running Database Migrations

Apply all migrations:

```
alembic upgrade head
```

Create new migration:

```
alembic revision --autogenerate -m "migration message"
```

Rollback migration:

```
alembic downgrade -1
```

9. Booking Conflict Prevention

The scheduling system prevents conflicting bookings using database-level mechanisms.

The system uses **PostgreSQL range indexing with GiST indexes** to ensure:

- two bookings cannot overlap for the same system
- scheduling conflicts are detected efficiently

This provides strong guarantees even under concurrent booking requests.

10. Installation

Prerequisites

Ensure the following are installed:

- Python 3.12+
- PostgreSQL
- `uv` (install guide)

Setup Environment

Navigate to the backend directory:

```
cd backend
uv sync
```

11. Running the Backend

Start the development server:

```
cd backend
export DATABASE_URL=postgresql+psycopg://crag:crag@localhost:5432/crag
export PYTHONPATH=src
uv run alembic upgrade head
uv run uvicorn crag.main:app --reload --app-dir src --host 0.0.0.0 --port 8000
```

The API will be available at:

```
http://localhost:8000
```

API documentation is available at:

```
http://localhost:8000/docs
```

12. Docker Setup (Optional)

If using Docker:

```
docker compose up --build
```

This starts:

- PostgreSQL database
 - backend service
 - optional frontend service
-

13. Development Workflow

Typical workflow:

1. Update database models
 2. Generate migration
 3. Apply migration
 4. Implement service logic
 5. Expose API endpoint
 6. Connect frontend
-

14. Logging and Observability

The backend logs:

- booking operations
- authentication activity
- scheduling conflicts
- audit generation

Future improvements may include:

- centralized logging
 - monitoring dashboards
 - resource utilization analytics
-

15. Security Considerations

Security mechanisms implemented:

- hashed password storage
- token-based authentication
- dependency-based authorization
- restricted access to administrative endpoints

Future improvements may include:

- role-based access policies
 - audit verification mechanisms
 - system-level resource enforcement
-

16. Future Extensions

Planned backend improvements include:

- advanced scheduling heuristics
- compute fairness metrics
- governance dashboards
- predictive resource allocation
- integration with compute cluster managers

17. Contribution Guidelines

When contributing to the backend:

- Maintain modular architecture
- Keep business logic inside service layers
- Avoid database logic inside routers
- Ensure migrations accompany schema changes
- Document new APIs

18. License

This project is distributed under the license defined in the repository root.

19. Maintainers

CRAGS Development Team

Frontend Guide

CRAGS Frontend

Compute Resource Allocation and Governance System (CRAGS)

This directory contains the **frontend application** for CRAGS. It provides the user interface through which users log in, view compute systems, create and modify resource bookings, manage group quotas, review the audit trail, and inspect usage analytics.

The frontend is a single-page React application that communicates with the **FastAPI backend** over REST. In production it is built to static assets and served by Nginx, which proxies `/api` to the backend.

1. Technology Stack

Component	Technology
Framework	React 18

Component	Technology
Language	TypeScript
Build Tool	Vite
UI Components	Material UI (MUI) v7
Charts	Recharts
Routing	React Router
API Communication	Axios (<code>api/client.ts</code>)
Data Fetching	React Query (<code>@tanstack/react-query</code> v5)
App State	React Query cache + React Context (theme)
Linting	ESLint + TypeScript ESLint
Package Manager	npm

There is no Redux in the project; server state lives in the React Query cache and the only global UI state (theme mode) lives in a React context.

2. Directory Structure

```

frontend/
|
├─ index.html          # Loads Roboto + Roboto Mono web fonts
├─ package.json
├─ vite.config.ts
├─ eslint.config.js
|
├─ src/
|   └─ main.tsx          # React root: QueryClient, BrowserRouter, AppThemeProvider
|   └─ App.tsx           # Route table; role-gated routes via RequireRole
|   └─ index.css         # Minimal global styles
|   |
|   └─ api/
|       └─ client.ts      # Axios instance: baseURL, withCredentials, 401 auto-refresh
|       └─ cragsApi.ts    # Typed wrappers for every backend endpoint
|       └─ api.ts         # Thin legacy re-export helpers
|   |
|   └─ hooks/            # React Query hooks (one concern per file)
|       └─ useCurrentUserQuery.ts
|       └─ useSystemsQuery.ts
|       └─ useBookingsQuery.ts
|       └─ useAvailabilityQuery.ts
|       └─ useAuditTrailQuery.ts
|       └─ useCreateBooking.ts
|       └─ useCreateSystem.ts
|       └─ useLoginMutation.ts
|       └─ useLogoutMutation.ts
|       └─ useSessionExpiry.ts
|       └─ shared/useDebounceValue.ts
|   |
|   └─ context/
|       └─ ThemeContext.tsx # Light/dark mode, persisted to localStorage
|   |
|   └─ theme/
|       └─ index.ts        # Centralised design system (see §7)
|   |
|   └─ pages/            # One file per route
|       └─ DashboardPage.tsx
|       └─ SchedulerPage.tsx
|       └─ SystemsPage.tsx
|       └─ MonitoringPage.tsx
|       └─ AnalyticsPage.tsx
|       └─ TeamPage.tsx
|       └─ LoginPage.tsx
|   |
|   └─ components/
|       └─ layout/        # AppShell, Sidebar (collapsible), TopBar
|       └─ panels/        # MissionControlDashboard, ApprovalQueuePanel,
|                           # WaitlistPanel, MaintenanceWindowsPanel,
|                           # SystemInventoryPanel, AuditTrailPanel,
|                           # BookingLifecycle, BookingActionsPanel, ...
|       └─ forms/         # BookingRequestForm, SystemRegistrationForm
|       └─ charts/        # ResourceConstraintChart, TemporalGantt
|       └─ ErrorBoundary.tsx
|   |
|   └─ lib/              # explainableError, policy, time helpers
|   └─ utils/            # simulateBooking (client-side booking preview)
|   └─ types/crags.ts     # Shared TypeScript types for all API shapes

```

3. Application Shell and Routing

`main.tsx` mounts the React root with the `QueryClientProvider`, `BrowserRouter`, and the theme provider. `App.tsx` defines the route table and wraps protected routes in a `RequireRole` guard that checks the current user's role before rendering.

`components/layout/AppShell.tsx` is the authenticated frame: a collapsible `Sidebar`, a `TopBar`, and the routed page content. On logout (or a `crag-session-expired` event) it clears the React Query cache and navigates to `/login`.

Route	Access	Page
<code>/</code>	Any authenticated	Dashboard
<code>/scheduler</code>	Any authenticated	Scheduler
<code>/systems</code>	RESOURCE_ADMIN, SUPER_ADMIN	Systems
<code>/monitoring</code>	GROUP_LEAD and above	Monitoring
<code>/analytics</code>	GROUP_LEAD and above	Analytics
<code>/team</code>	RESOURCE_ADMIN, SUPER_ADMIN	Team
<code>/login</code>	Public	Login

4. Data Flow

Component → React Query hook (src/hooks) → `cragApi.ts` → Axios client → backend

- Components call **hooks**, never `cragApi.ts` or Axios directly. This keeps caching, loading/error states, and invalidation centralised.
- `api/client.ts` sets `baseUrl` from `VITE_API_BASE_URL` (default `/api/v1`) and enables `withCredentials` so the HTTP-only session cookie is sent.
- On a `401`, the client's interceptor calls `/auth/refresh` once; if that also fails it dispatches a `crag-session-expired` DOM event, which the shell handles by clearing the cache and redirecting to login.

Note: because `baseUrl` already includes `/api/v1`, endpoint paths in `cragApi.ts` must **not** repeat that prefix.

5. Installation

Prerequisites

- Node.js 18 or later
- npm

```
node -v
npm -v
```

Install Dependencies

```
cd frontend
npm install
```

6. Running, Building, and Linting

Development server (start the backend first):

```
npm run dev      # http://localhost:5173, proxies /api to http://localhost:8000
```

Production build (also runs `tsc`):

```
npm run build    # output to frontend/dist; Nginx serves this in Docker
```

Linting and type checking:

```
npm run lint
npx tsc --noEmit
```

7. Design System

All colour, typography, and shape live in `src/theme/index.ts`, the single source of truth for the UI. To restyle the app, edit this file rather than individual components:

- `buildTheme(mode)` — MUI theme factory for light/dark. Flat bordered cards (no elevation), 8px radius, no gradients or glow effects (Google Workspace style).
- `BRAND` — the Material palette (`blue #1A73E8`, `green #1E8E3E`, `amber #F9AB00`, `red #D93025`, `purple #9334E6`, `teal #12A4AF`).
- `STATUS_COLOR` / `statusTone(status)` — semantic `{ color, bg }` tones for booking and system statuses; panels call `statusTone()` instead of hardcoding hex values.
- `CHART_COLORS` — ordered series colours for Recharts views.
- `FONT_SANS` / `FONT_MONO` — Roboto and Roboto Mono, loaded in `index.html` .

Theme mode is persisted to `localStorage` via `context/ThemeContext.tsx` and defaults to light. Components should consume semantic tokens (`primary.main`, `text.secondary`, `action.hover`, `divider`) rather than literal colours.

8. Contribution Guidelines

- Keep shared types in `src/types/crags.ts` .
- Add new API calls to `src/api/cragsApi.ts` and wrap them in a React Query hook under `src/hooks/` .
- Do not embed backend URLs or raw Axios calls inside components.
- Use design-system tokens from `src/theme/index.ts` ; do not introduce ad-hoc colours or fonts.
- Document non-obvious components and hooks with a brief comment.

9. License

This project is distributed under the license specified in the root repository.